6CCSACCM

Cooperation, Coordination and Multi-Agent Systems

Revision Notes

Academic Year 2025/26 — Semester 2

Compiled on April 2, 2026

These notes are structured to help you prepare for exams.
Each week includes: learning objectives, core concepts, theory, and worked examples.

Contents

Module Overview **5**

1 Week 1: Non-Cooperative Games (Part I) **7**
1.1 Topic Summary . . . 7
1.2 Overview + Core Concepts + Extended Theory . . . 8
1.2.1 Multi-Agent Systems . . . 8
1.2.2 Normal Form Games . . . 8
1.2.3 The Prisoner's Dilemma . . . 9
1.2.4 Dominance . . . 10
1.2.5 Iterated Elimination of Dominated Strategies (IEDS) . . . 10
1.2.6 Nash Equilibrium (Pure Strategies) . . . 11
1.2.7 Coordination Games . . . 12
1.2.8 Mixed Strategies . . . 12
1.2.9 Computing Mixed Strategy Nash Equilibria . . . 13
1.2.10 Maximin and Minimax Strategies . . . 14
1.2.11 Zero-Sum Games and the Minimax Theorem . . . 15
1.2.12 Other Solution Concepts . . . 16
1.2.13 Summary: Classic Games . . . 16
1.2.14 Worked Examples from Exercises . . . 17
1.2.15 Other Game Types (Preview) . . . 17

2 Week 2: Non-Cooperative Games (Part II) **19**
2.1 Topic Summary . . . 19
2.2 Overview + Core Concepts + Extended Theory . . . 20
2.2.1 Large Games and Representation . . . 20
2.2.2 Congestion Games . . . 20
2.2.3 Network Congestion (Routing) Games . . . 21
2.2.4 Applications of Congestion Games . . . 22
2.2.5 Best Response Dynamics . . . 23
2.2.6 Rosenthal's Potential Function . . . 24
2.2.7 Potential Games . . . 25
2.2.8 Price of Anarchy and Price of Stability . . . 25
2.2.9 Linear Congestion Games . . . 27
2.2.10 Relating Potential Function to Social Cost . . . 28
2.2.11 Worked Examples from Exercises . . . 29
2.2.12 Summary: Key Concepts . . . 31

3 Week 3: Repeated and Extensive-Form Games **32**
3.1 Topic Summary . . . 32
3.2 Overview + Core Concepts + Extended Theory . . . 33
3.2.1 From Normal Form to Extensive Form . . . 33

3.2.2 Perfect-Information Extensive Form Games . . . 33
3.2.3 Strategies in Perfect-Information Games . . . 34
3.2.4 Extensive Form vs. Normal Form . . . 35
3.2.5 Subgame-Perfect Equilibrium . . . 35
3.2.6 Computing SPE: Backward Induction . . . 36
3.2.7 Imperfect-Information Games . . . 36
3.2.8 Repeated Games . . . 37
3.2.9 Finitely Repeated Games . . . 38
3.2.10 Infinitely Repeated Games . . . 38
3.2.11 Sustaining Cooperation: Tit-for-Tat and Trigger . . . 38
3.2.12 Summary: Key Concepts . . . 40

4 Week 4: Cooperative Games (Part I) 41
4.1 Topic Summary . . . 41
4.2 Overview + Core Concepts + Extended Theory . . . 42
4.2.1 From Non-Cooperative to Cooperative Games . . . 42
4.2.2 Transferable Utility Games: Formal Definition . . . 43
4.2.3 Important Game Types . . . 43
4.2.4 Special Classes of TU Games . . . 44
4.2.5 The Core . . . 45
4.2.6 Core of Special Game Classes . . . 46
4.2.7 Core Relaxations . . . 47
4.2.8 Shapley Value . . . 47
4.2.9 Shapley Value: Axiomatic Properties . . . 48
4.2.10 Banzhaf Index . . . 49
4.2.11 Summary: Key Concepts . . . 50

5 Week 5: Cooperative Games (Part II) 51
5.1 Topic Summary . . . 51
5.2 Overview + Core Concepts + Extended Theory . . . 52
5.2.1 Non-Transferable Utility Games . . . 52
5.2.2 Hedonic Games . . . 52
5.2.3 Compactly Representable Hedonic Games . . . 53
5.2.4 Stability Concepts for Hedonic Games . . . 53
5.2.5 Friends-and-Enemies Games . . . 55
5.2.6 The Stable Matching Problem . . . 55
5.2.7 The Stable Roommate Problem . . . 56
5.2.8 Gale-Shapley Deferred Acceptance Algorithm . . . 57
5.2.9 Optimality and Pessimality . . . 57
5.2.10 Strategyproofness . . . 58
5.2.11 Applications and Extensions . . . 58
5.2.12 Worked Examples from Exercises . . . 59
5.2.13 Summary: Key Concepts . . . 61

6 Week 7: Fair Division (Part I) 62
6.1 Topic Summary . . . 62
6.2 Overview + Core Concepts + Extended Theory . . . 62
6.2.1 The Fair Division Setting . . . 63
6.2.2 Fairness Criteria . . . 63
6.2.3 Cake-Cutting Model . . . 64
6.2.4 Cut-and-Choose (2 Agents) . . . 64
6.2.5 Dubins–Spanier Moving Knife (n Agents, Proportional) . . . 65
6.2.6 Even–Paz Algorithm (n Agents, Proportional, Optimal) . . . 66
6.2.7 Selfridge–Conway Algorithm (3 Agents, Envy-Free) . . . 66
6.2.8 General Envy-Free Cake Cutting . . . 67
6.2.9 Worked Examples . . . 68

7 Week 8: Fair Division (Part II) 71
7.1 Topic Summary . . . 71
7.2 Overview + Core Concepts + Extended Theory . . . 71
7.2.1 Indivisible Goods: Model and Applications . . . 72
7.2.2 Fairness Relaxations: EF1 and PROP1 . . . 72
7.2.3 Round-Robin Algorithm (Additive Valuations) . . . 73
7.2.4 Limitations of Round-Robin . . . 74
7.2.5 Envy-Cycle Elimination (Monotone Valuations) . . . 74
7.2.6 Concepts of Welfare . . . 75
7.2.7 Fairness vs. Efficiency Trade-Offs . . . 75
7.2.8 Envy-Freeness up to Any Good (EFX) . . . 76
7.2.9 Worked Examples from Exercises . . . 77
7.2.10 Summary: Algorithms and Guarantees . . . 78

Module Overview

About This Module

This module focuses on **multi-agent systems**—systems consisting of multiple autonomous entities with distributed information, computational capabilities, and potentially conflicting objectives. These systems can be:

- **Cooperative**: sensor networks, teams of mobile robots in a warehouse
- **Competitive**: electronic marketplaces, resource/task allocation
- **Mixed**: collaborative platforms like Wikipedia or Stack Overflow

Agents may be entirely artificial, entirely human, or a mix of both.

Module code: 6CCSACCM **Semester:** 2 (AY 2025/26)

Module Aims

The module aims to:

- Discuss foundational and recent research on various aspects of **multi-agent systems**.
- Place particular emphasis on **coordination strategies** and **fostering cooperative behaviour**.
- Focus on **networks** and the **internet economy**.
- Introduce students to the field and help identify **potential research directions**.
- Develop skills in **critical analysis of research papers** and **formulating research questions**.

Learning Outcomes

By the end of this module, you will be able to:

Code	Learning Outcome
CCM1	Use **game-theoretic, algorithmic and computational tools** to model and analyse multi-agent strategic behaviour and its outcomes.
CCM2	Use **game-theoretic tools** to foster cooperation and stability in multi-agent systems.
CCM3	Use **algorithmic mechanism design techniques** to model and design incentives in multi-agent interaction.
CCM4	Apply **computational social choice techniques** to analyse multi-agent decision-making and improve the quality and stability of collective decisions.

Key Topics (Expected)

Based on the learning outcomes, this module is likely to cover:

Area	Topics
Game Theory	Strategic games, Nash equilibrium, coordination games, Prisoner's Dilemma, repeated games
Cooperation	Mechanisms for fostering cooperation, stability concepts, coalition formation
Mechanism Design	Incentive design, auctions, voting mechanisms, truthfulness
Social Choice	Voting rules, Arrow's theorem, manipulation, collective decision-making
Networks	Network effects, internet economy, platform design

Assessment and Feedback

Formative assessment opportunities:

- **Discussion exercises**: peer discussion and feedback
- **Tutorial questions**: regular problem sheets with feedback during sessions
- **Discussion forums**: post questions on KEATS
- **Office hours**: direct feedback opportunities

[TODO] Add assessment structure (exam format, coursework) when details are provided.

---

1 Week 1: Non-Cooperative Games (Part I)

1.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Define the core concepts of non-cooperative game theory, including **strategy profiles**, **payoff functions**, and the distinction between cooperative and non-cooperative games.
2. Formulate real-world scenarios as **strategic-form (normal-form)** or extensive-form games involving rational, self-interested agents.
3. Identify and classify various types of strategies: **pure**, **mixed**, **dominant**, and **best response** strategies.
4. Analyse game-theoretic models to determine the existence and properties of **Nash equilibria** (in pure and mixed strategies).
5. Explain and interpret equilibrium behaviour in classical games: **Prisoner's Dilemma**, **Matching Pennies**, **Battle of the Sexes**, **Stag Hunt**, and coordination games.
6. Evaluate the potential for and limitations of non-cooperative solutions, including issues of **uniqueness**, **multiplicity**, and **inefficiency** of equilibria.
7. Demonstrate competency in using mathematical notation and reasoning to represent and solve strategic games.

The Big Picture

This week introduces **non-cooperative game theory**—the mathematical framework for analysing strategic interactions between rational decision-makers. The module sits at the intersection of two fields:

- **Algorithmic Game Theory (AGT):** Studies algorithms in strategic environments where inputs come from self-interested agents. Combines theoretical CS and economics to analyse equilibria and design efficient, fair mechanisms.
- **Computational Social Choice (COMSOC):** Studies collective decision-making and preference aggregation, asking how individual preferences can be combined into collective decisions.

The central question: *"What is a stable outcome when multiple self-interested agents interact?"*

What Examiners Like to Ask

- Define a game in **normal form**; write the formal tuple $\Gamma = (N, (S_i), (U_i))$.
- Given a payoff matrix, identify **dominated strategies** and solve by iterated elimination.
- Find all **pure strategy Nash equilibria** using best-response analysis.
- Compute **mixed strategy Nash equilibria** using the indifference principle.
- Compare **maximin** vs **minimax** strategies; apply the Minimax theorem to zero-sum games.
- Explain why Prisoner's Dilemma has a unique NE that is Pareto-inefficient.
- Distinguish **Nash equilibrium** from **strong equilibrium**.

1.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Multi-agent systems and game types
2. Normal form games: formal definition
3. Dominance and iterated elimination
4. Nash equilibrium in pure strategies
5. Mixed strategies and expected payoffs
6. Nash equilibrium in mixed strategies
7. Computing mixed NE (indifference principle)
8. Maximin and minimax strategies
9. Zero-sum games and the Minimax theorem
10. Other solution concepts (strong equilibrium, correlated equilibrium)

1.2.1 Multi-Agent Systems

Multi-Agent System (MAS)

A **multi-agent system** consists of multiple autonomous entities (agents) with:

- Distributed information
- Computational capabilities
- Potentially conflicting objectives

Types:

Type	Examples
Cooperative	Sensor networks, robot teams in warehouses
Competitive	Electronic marketplaces, resource allocation
Mixed	Wikipedia, Stack Overflow

1.2.2 Normal Form Games

Non-Cooperative Game

In a **non-cooperative** (or competitive) game:

- Players take actions and enjoy outcomes *individually*
- There are **no binding contracts** among players
- Each player strategically optimises their own utility function

Normal Form Game (Formal Definition)

A game in **strategic (normal) form** is a tuple:

$$\Gamma = (N, (S_i)_{i \in N}, (U_i)_{i \in N})$$

where:

- N is a finite set of **players**
- S_i is the set of **strategies** for player $i \in N$
- $U_i : S \to \mathbb{R}$ is the **utility function** for player i, where $S = \times_{i \in N} S_i$ is the set of all strategy profiles

Strategy Profile Notation

A **profile** s is a vector of strategies $(s_1, \ldots, s_n)$ where $s_i \in S_i$.

Key notation:

- $s_{-i} = (s_1, \ldots, s_{i-1}, s_{i+1}, \ldots, s_n)$ — profile without player i's strategy
- $s = (s_i, s_{-i})$ — decomposition of profile
- s_C — strategies of coalition $C \subseteq N$
- s_{-C} — strategies of players not in C

Exam Tip: Writing Game Definitions

When asked to "formulate as a game", always specify:

1. The set of players N
2. The strategy set S_i for each player
3. The utility function U_i (or payoff matrix for finite games)

1.2.3 The Prisoner's Dilemma

Prisoner's Dilemma

Setup: Two suspects are interrogated in separate rooms. Each can:

- **Cooperate (C):** Deny the crime
- **Defect (D):** Confess (rat out the other)

Payoff matrix:

	C	D
C	$-1, -1$	$-10, 0$
D	$0, -10$	$-5, -5$

Reading the matrix: First number = row player's payoff; second = column player's.

Key insight: The only stable outcome is (D, D), even though (C, C) gives both players a better payoff!

Prisoner's Dilemma: General Form

	C	D
C	R, R	S, T
D	T, S	P, P

Condition: $T > R > P > S$

- T = Temptation (defect while other cooperates)
- R = Reward (mutual cooperation)
- P = Punishment (mutual defection)
- S = Sucker's payoff (cooperate while other defects)

1.2.4 Dominance

Strict and Weak Dominance

For player i and strategies $s, t \in S_i$:

Strict dominance: s ***strictly dominates*** t if:

$$\forall s_{-i} \in S_{-i} : \quad U_i(s, s_{-i}) > U_i(t, s_{-i})$$

Weak dominance: s ***weakly dominates*** t if:

$$\forall s_{-i} \in S_{-i} : U_i(s, s_{-i}) \geq U_i(t, s_{-i}) \quad \text{and} \quad \exists s'_{-i} : U_i(s, s'_{-i}) > U_i(t, s'_{-i})$$

Dominant and Dominated Strategies

- Strategy s is **dominant** if it dominates *all* other strategies
- Strategy t is **dominated** if *some* other strategy dominates it

Key principle: Rational players never choose strictly dominated strategies.

Exam Tip: Dominance is Rare but Powerful

Dominant strategy is a very strong solution concept—no assumptions needed about other players. However, it rarely exists. In Prisoner's Dilemma, Defect is dominant for both players.

1.2.5 Iterated Elimination of Dominated Strategies (IEDS)

IEDS Algorithm

Procedure:

1. Find a strictly dominated strategy for any player
2. Eliminate it from the game
3. Repeat until no dominated strategies remain

Properties:

- **Strict dominance:** Order-independent (same final result)
- **Weak dominance:** Order-dependent (different elimination orders may give different results)
- Surviving strategies need not be dominant
- May end with multiple strategies per player

Example: IEDS on a 3×3 Game

Original game:

	L	C	R
T	$73, 25$	$57, 42$	$66, 32$
M	$80, 26$	$35, 12$	$32, 54$
B	$28, 27$	$63, 31$	$54, 29$

Step 1: For column player, R dominates L (compare: $32 > 25$, $54 > 26$, $29 > 27$). Eliminate L.

Step 2: For row player, T and B both dominate M. Eliminate M.

Step 3: For column player, C dominates R. Eliminate R.

Step 4: For row player, B dominates T ($63 > 57$). Eliminate T.

Solution: (B, C) with payoffs $(63, 31)$.

Verify NE: Row player: $63 > 54$ (B vs T against C). Column player: $31 > 29$ (C vs R against B). ✓

1.2.6 Nash Equilibrium (Pure Strategies)

Nash Equilibrium

A strategy profile $s^* = (s_1^*, \ldots, s_n^*)$ is a **(pure strategy) Nash equilibrium** if no player has a **unilateral improving move**:

$$\forall i \in N, \forall s_i' \in S_i : \quad U_i(s_i^*, s_{-i}^*) \geq U_i(s_i', s_{-i}^*)$$

Equivalently: each player plays a **best response** to the others' strategies.

Best Response

Strategy s_i is a **best response** to s_{-i} if:

$$s_i \in \arg\max_{s_i' \in S_i} U_i(s_i', s_{-i})$$

Exam Tip: Finding Pure Strategy NE

Method 1 (Best Response):

1. For each opponent strategy, find player's best response (underline/mark it)
2. NE = cells where *all* players are playing best responses

Method 2 (Check deviations):

1. For each cell, check if any player wants to deviate unilaterally
2. NE = cells with no profitable deviations

Example: Best Response Analysis

	L	C	R
T	$2, 2$	$1^*, 3^*$	$0^*, 1$
M	$3^*, 1^*$	$0, 0$	$0^*, 0$
B	$1, 0^*$	$0, 0^*$	$0^*, 0^*$

Reading: * marks best responses. NE = cells where both payoffs are marked.

Nash equilibria: None! No cell has both payoffs marked.

1.2.7 Coordination Games

Coordination Game

A game where players benefit from **choosing corresponding strategies**:

	Left	Right
Up	$1, 1$	$0, 0$
Down	$0, 0$	$1, 1$

Properties:

- **Two pure strategy NE:** (Up, Left) and (Down, Right)
- No dominant or dominated strategies
- Both NE are also **strong equilibria**

Battle of the Sexes

Players want to coordinate but have **different preferences**:

	Theater	Football
Theater	$2, 1$	$0, 0$
Football	$0, 0$	$1, 2$

Pure NE: (Theater, Theater) and (Football, Football)

Stag Hunt

	Stag	Hare
Stag	$4, 4$	$1, 3$
Hare	$3, 1$	$3, 3$

Story: Two hunters can cooperate for a stag (high reward) or safely catch a hare alone.

Pure NE: (Stag, Stag) and (Hare, Hare)—plus mixed equilibria (see below).

1.2.8 Mixed Strategies

Mixed Strategy

A **mixed strategy** is a probability distribution over pure strategies.

- $\Delta_i = \Delta(S_i)$ = set of probability distributions over S_i
- A pure strategy is a mixed strategy with probability 1 on one action
- Even with finite S_i, there are infinitely many mixed strategies

Expected Utility

When mixed strategy profile $\tilde{s} \in \Delta$ is played:

$$U_i(\tilde{s}) = \sum_{s \in S} U_i(s) \cdot \Pr(s|\tilde{s})$$

where:

$$\Pr(s|\tilde{s}) = \prod_{j \in N} \tilde{s}_j(s_j)$$

(product of individual probabilities, assuming independence)

Nash's Existence Theorem (1951)

Every finite game (finite players, finite action sets) has at least one **mixed strategy Nash equilibrium**.

1.2.9 Computing Mixed Strategy Nash Equilibria

Indifference Principle

In a mixed strategy NE where player i randomises over actions $A \subseteq S_i$:

Player i must be indifferent among all actions in A:

$$\forall a, a' \in A: \quad U_i(a, \tilde{s}_{-i}) = U_i(a', \tilde{s}_{-i})$$

Key insight: The *other* player's mixing probabilities make you indifferent!

Example: Matching Pennies

	H	T
H	$1, -1$	$-1, 1$
T	$-1, 1$	$1, -1$

No pure strategy NE (verify: every cell has a profitable deviation).

Mixed NE: Let Player 1 play H with probability p, Player 2 play H with probability q.

Player 2's indifference:

$$U_2(\text{H}) = U_2(\text{T}) \implies -p + (1-p) = p - (1-p) \implies p = \frac{1}{2}$$

By symmetry: $q = \frac{1}{2}$

Mixed NE: $\left(\frac{1}{2}, \frac{1}{2}\right), \left(\frac{1}{2}, \frac{1}{2}\right)$ with expected utility 0 for both.

Example: Stag Hunt Mixed NE

	Stag	Hare
Stag	$4, 4$	$1, 3$
Hare	$3, 1$	$3, 3$

Let $p = \Pr(\text{Player 1 plays Stag})$, $q = \Pr(\text{Player 2 plays Stag})$.

Player 2's indifference:

$$U_2(\text{Stag}) = U_2(\text{Hare}) \implies 4p + 1(1-p) = 3p + 3(1-p)$$

$$4p + 1 - p = 3p + 3 - 3p \implies 3p + 1 = 3 \implies p = \frac{2}{3}$$

By symmetry: $q = \frac{2}{3}$

Three NE total: Two pure + one mixed at $\left(\frac{2}{3}, \frac{2}{3}\right)$.

Exam Tip: Mixed NE Computation

1. Set up probability variables (p, q, etc.)
2. Write expected utility equations for each action
3. Set utilities equal (indifference condition)
4. Solve for the *other* player's probability
5. Verify the solution makes sense (probabilities in $[0, 1]$)

Common error: Solving for your own probability instead of the opponent's!

1.2.10 Maximin and Minimax Strategies

Maximin (Safety Level) Strategy

The **maximin strategy** for player i maximises their worst-case payoff:

$$s_i^{LS} \in \arg\max_{s_i} \min_{s_{-i}} U_i(s_i, s_{-i})$$

The **maximin value** (safety level) is:

$$U_i^{LS} = \max_{s_i} \min_{s_{-i}} U_i(s_i, s_{-i})$$

Intuition: If opponents play to hurt you as much as possible, what's your best guarantee?

Minimax Strategy (2-Player Games)

The **minimax strategy** for player i against player $-i$ minimises $-i$'s maximum payoff:

$$s_i^{MM} \in \arg\min_{s_i} \max_{s_{-i}} U_{-i}(s_i, s_{-i})$$

Intuition: How can you punish the opponent most, ignoring your own payoff?

Maximin in Prisoner's Dilemma

	C	D
C	$-1, -1$	$-10, 0$
D	$0, -10$	$-5, -5$

Row player's analysis:

- If commit to C: opponent plays D (minimises row), payoff $= -10$
- If commit to D: opponent plays D (minimises row), payoff $= -5$

Maximin: Choose D (guarantees $-5 > -10$)

Defect is both the dominant strategy AND the maximin strategy.

1.2.11 Zero-Sum Games and the Minimax Theorem

Constant-Sum and Zero-Sum Games

A two-player game is **constant-sum** if $\exists c$ such that:

$$\forall s \in S_1 \times S_2 : \quad U_1(s) + U_2(s) = c$$

For $c = 0$: **zero-sum game** (pure conflict—one player's gain is the other's loss).

Minimax Theorem (von Neumann, 1928)

For any finite, two-player, zero-sum game:

$$\max_{s_1} \min_{s_2} U_1(s_1, s_2) = \min_{s_2} \max_{s_1} U_1(s_1, s_2)$$

Consequences:

1. Each player's **maximin value = minimax value**
2. The common value is called the **value of the game**
3. **Maximin strategies = minimax strategies = NE strategies**
4. All Nash equilibria have the **same payoff vector**

Matching Pennies: Minimax Analysis

	H	T
H	$1, -1$	$-1, 1$
T	$-1, 1$	$1, -1$

This is a zero-sum game ($U_1 + U_2 = 0$ always).

Value of the game: 0

By Minimax theorem: The unique NE $\left(\frac{1}{2}, \frac{1}{2}\right)$ is also the maximin = minimax strategy for each player.

Computing Maximin/Minimax in General-Sum Games

For a general-sum 2-player game:

1. Player i's maximin depends *only* on i's utilities
2. Create a zero-sum game: keep i's utilities, negate them for $-i$
3. Solve the zero-sum game for NE
4. NE strategies = maximin strategies

1.2.12 Other Solution Concepts

Strong Equilibrium

A profile s is a **strong equilibrium** if no coalition $C \subseteq N$ has a joint improving move:

$$\nexists C \subseteq N, s'_C \in S_C : \quad \forall i \in C : U_i(s'_C, s_{-C}) > U_i(s)$$

Key facts:

- Every strong equilibrium is a Nash equilibrium (consider singleton coalitions)
- Much more rare than NE
- Prisoner's Dilemma has **no** strong equilibrium
- Coordination game: both pure NE are strong equilibria

Correlated Equilibrium

Players condition actions on a shared random signal from a trusted mediator.

- Every mixed NE induces a correlated equilibrium
- Set of correlated equilibria $\supseteq$ set of NE
- **Computationally easier:** Can be found via linear programming

ε-Nash Equilibrium

A profile where no player can gain more than $\varepsilon > 0$ by deviating:

$$\forall i, \forall s'_i : \quad U_i(s_i, s_{-i}) \geq U_i(s'_i, s_{-i}) - \varepsilon$$

Standard NE is the special case $\varepsilon = 0$.

Rationalizable Strategies

A strategy is **rationalizable** if it is a best response to some belief about opponents' strategies (consistent with common knowledge of rationality).

- Equivalent to surviving iterated elimination of never-best-responses
- Every NE strategy is rationalizable
- Set of rationalizable strategies $\supseteq$ NE strategies

1.2.13 Summary: Classic Games

Game	Pure NE	Mixed NE	Strong NE	Key Feature
Prisoner's Dilemma	1 (D,D)	Same	None	Dominant strategy, Pareto-inefficient
Coordination	2	+1	2	Multiple equilibria
Matching Pennies	0	1	0	Zero-sum, no pure NE
Battle of Sexes	2	+1	2	Asymmetric preferences
Stag Hunt	2	+1	2	Risk vs reward trade-off

1.2.14 Worked Examples from Exercises

Exercise: Dividing Money

Setup: Two players divide £10. Each names $x_i \in \{0, 1, \ldots, 10\}$.

- If $x_1 + x_2 \leq 10$: each gets their claim (remainder destroyed)
- If $x_1 + x_2 > 10$ and $x_1 \neq x_2$: lower claimant gets x_i, other gets rest
- If $x_1 + x_2 > 10$ and $x_1 = x_2$: each gets £5

Best responses:

Other's claim	Best responses
0	$\{10\}$
1	$\{9, 10\}$
2	$\{8, 9, 10\}$
$\vdots$	$\vdots$
5	$\{5, 6, 7, 8, 9, 10\}$
6	$\{5, 6\}$
7	$\{6\}$
8	$\{7\}$
9	$\{8\}$
10	$\{9\}$

Nash equilibria: $(5, 5)$, $(5, 6)$, $(6, 5)$, $(6, 6)$ — any pair where both are best responses to each other.

Exercise: Modified Stag Hunt

Modified payoffs:

	Stag	Hare
Stag	$4, 4$	$1, 3$
Hare	$3, 2$	$3, 3$

Whose mixing probability changes?

Player 2's utility changed ($1 \to 2$ in Hare-Stag cell).

Effect: Player 2's utility is used to determine Player 1's indifference condition.

Player 1's mixing probability changes: $p = \frac{1}{2}$ (was $\frac{2}{3}$).

Player 2's mixing probability stays: $q = \frac{2}{3}$ (Player 1's utilities unchanged).

1.2.15 Other Game Types (Preview)

Game Type	Description
Extensive form	Game tree; players move in turns (sequential)
Repeated games	Same game played multiple times; cumulative payoffs
Cooperative (coalitional)	Players form coalitions; binding contracts; characteristic function over coalitions

These will be covered in subsequent weeks.

2 Week 2: Non-Cooperative Games (Part II)

2.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Define the core concepts of non-cooperative game theory, including **strategy profiles**, **payoff functions**, and the distinction between cooperative and non-cooperative games.
2. Formulate real-world scenarios as **strategic-form (normal-form)** or extensive-form games involving rational, self-interested agents.
3. Identify and classify various types of strategies: **pure**, **mixed**, **dominant**, and **best response** strategies.
4. Analyse game-theoretic models to determine the existence and properties of **Nash equilibria** (in pure and mixed strategies) and other solution concepts.
5. Explain and interpret equilibrium behaviour in classical games such as the Prisoner's Dilemma, Matching Pennies, Battle of the Sexes, and coordination games.
6. Evaluate the potential for and limitations of non-cooperative solutions, including issues of **uniqueness**, **multiplicity**, and **inefficiency** of equilibria.
7. Demonstrate competency in using mathematical notation and reasoning to represent and solve strategic games.

The Big Picture

This week explores how strategic interactions evolve over time (**equilibrium dynamics**) and investigates the **efficiency of equilibria**. We study:

- **Congestion games:** A structured class of large games with concise representation
- **Best-response dynamics:** How agents naturally converge to equilibria
- **Potential functions:** A powerful tool for proving equilibrium existence
- **Price of Anarchy/Stability:** Quantifying the cost of selfish behaviour

The central question: *"How inefficient are equilibria compared to socially optimal outcomes?"*

What Examiners Like to Ask

- Define a **congestion game** formally; specify players, resources, actions, and cost functions.
- Given a network graph, compute player costs in different states and identify **Nash equilibria**.
- Explain why best-response dynamics converge in congestion games using **Rosenthal's potential function**.
- Calculate the **Price of Anarchy** and **Price of Stability** for given games.
- Prove bounds relating social cost to potential function values.
- Design games achieving specific PoA/PoS values (e.g., cut games with PoA = 2).

2.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Large games and representation complexity
2. Congestion games: formal definition
3. Network/routing congestion games
4. Applications of congestion games
5. Best-response dynamics
6. Rosenthal's potential function
7. Potential games and the finite improvement property
8. Price of Anarchy and Price of Stability
9. Linear congestion games and tight bounds

2.2.1 Large Games and Representation

Representation Complexity

To represent an n-player game where each player has k actions in normal form, we need $\mathbf{k^n}$ numbers just to encode the utility functions.

Problem: For even moderately large k and n, this is intractable.

Solution: Study games with **concise structure** that allows compact representation and efficient reasoning.

Exam Tip

When asked about scalability of game representations, emphasize that normal form requires exponential space in the number of players, motivating structured game classes like congestion games.

2.2.2 Congestion Games

Congestion Game (Formal Definition)

A **congestion game** is defined by the tuple $(N, R, (A_i)_{i \in N}, (d_j)_{j \in R})$ where:

- $N = \{1, 2, \ldots, n\}$ is the set of **players**
- $R = \{1, 2, \ldots, m\}$ is the set of **facilities** (or resources)
- For each player i, $A_i \subseteq 2^R$ is the set of **actions**, where each action $a_i \in A_i$ is a subset of facilities
- For each facility $j \in R$, $d_j : \{0, 1, \ldots, n\} \to \mathbb{R}_{\geq 0}$ is the **cost (delay) function**

Key insight: The cost of a facility depends *only* on how many players use it, not on which specific players.

Player Costs in Congestion Games

Given action profile $a = (a_1, \ldots, a_n)$:

Congestion on facility j:

$$n_j(a) = |\{i : j \in a_i\}|$$

(number of players using facility j)

Cost to player i:

$$c_i(a) = \sum_{j \in a_i} d_j(n_j(a))$$

(sum of delay costs over all facilities in player i's action)

Exam Tip: Congestion Games vs Normal Form

Congestion games have **polynomial representation**: $O(n \cdot |R| + m \cdot n)$ to specify all action sets and delay functions. Compare to $O(k^n)$ for general normal form!

2.2.3 Network Congestion (Routing) Games

Network Congestion Game

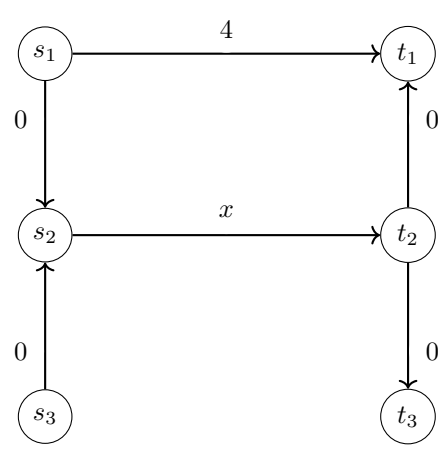
A **network congestion game** (or **routing game**) is a congestion game where:

- There is a directed graph $G = (V, E)$
- The resource set R corresponds to the edge set E
- Each player i has a source-destination pair (s_i, t_i)
- Player i's action set A_i = set of all directed paths from s_i to t_i

Interpretation: Each player chooses a route through the network; edge costs depend on congestion.

Example: Three-Player Routing Game

Consider the following network with three players:



Setup:

- Player 1: $s_1 \to t_1$ (two paths available)
- Player 2: $s_2 \to t_2$ (one path: direct)
- Player 3: $s_3 \to t_3$ (one path: via s_2, t_2)

Edge costs: Most edges have constant cost; edge (s_2, t_2) has cost $d(x) = x$ (identity function).

Player 1's strategies:

1. Direct path: $\{(s_1, t_1)\}$ with constant cost 4

2. Indirect path: $\{(s_1, s_2), (s_2, t_2), (t_2, t_1)\}$ with cost $0 + x + 0 = x$

State A (Player 1 takes direct route):

- Congestion on (s_2, t_2): 2 (Players 2 and 3)
- $c_1(A) = 4, \quad c_2(A) = 2, \quad c_3(A) = 2$
- Social cost: $4 + 2 + 2 = 8$

State B (Player 1 takes indirect route):

- Congestion on (s_2, t_2): 3 (all players)
- $c_1(B) = 3, \quad c_2(B) = 3, \quad c_3(B) = 3$
- Social cost: $3 + 3 + 3 = 9$

Analysis:

- **Nash Equilibrium:** State B (Player 1 prefers cost 3 over cost 4)
- **Social Optimum:** State A (total cost 8 vs 9)
- Selfish behaviour leads to **inefficient** equilibrium!

2.2.4 Applications of Congestion Games

Congestion games model decentralised decision-making in shared, congestible environments:

Domain	**Application**
Traffic & Transportation	Drivers choose routes to minimise travel time; analysis of road networks, navigation apps, highway design
Telecommunications	Data packets select routes/channels; TCP/IP routing, spectrum allocation, cellular load balancing
Cloud Computing	Jobs compete for CPUs, memory, bandwidth in data centres
Robotics	Multiple robots share warehouse aisles, coordinate movement
Smart Grids	Consumers choose when to use electricity; demand response
IoT Networks	Devices compete for transmission time, battery, sensors
Shared Economy	Ride-hailing, bike-sharing: too many users at one location reduces utility
AI/ML Resources	Competing agents select computing resources; federated learning
Cybersecurity	Attackers/defenders allocate resources across network surfaces
Urban Planning	Citizens choose public resources (parks, hospitals); overuse degrades quality

---

2.2.5 Best Response Dynamics

Best Response Dynamics

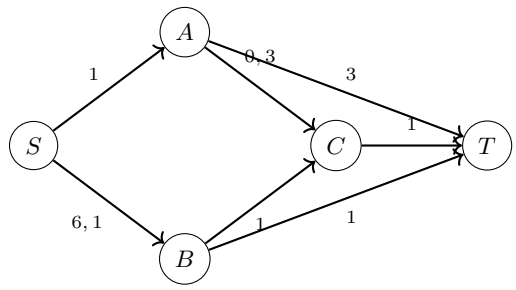
Algorithm:

1. Start with an arbitrary action profile $a = (a_1, \ldots, a_n)$
2. While $\exists$ player i who can decrease their cost by switching:
   - Player i switches to a **best response** (or any improving response)
3. If/when the process halts, return the current profile

Key observation: If best-response dynamics halts, the output is a **pure Nash equilibrium** (by definition: no player has a profitable deviation).

Example: Best Response Dynamics in a Routing Game

Consider two players (Red and Blue) travelling from S to T:



Edge labels show cost for 1 user, or (cost for 1, cost for 2) if congestion-dependent.

Initial state: Red takes upper path $S \to A \to T$, Blue takes lower path $S \to B \to T$.

- Red's cost: $1 + 0 + 3 = 4$
- Blue's cost: $6 + 1 = 7$

Round 1 (Red deviates): Red switches to $S \to A \to C \to T$ (cost: $1 + 0 + 1 = 2 < 4$).

Round 2 (Blue deviates): Blue switches to $S \to B \to C \to T$ (cost: $1 + 1 + 3 = 5 < 7$).

- Now both use edge (C, T), so Red's cost becomes $1 + 0 + 3 = 4$

Round 3 (Red deviates): Red switches back to $S \to A \to T$ (cost: $1 + 0 + 3 = 4$). Same cost, but let's say no strict improvement.

Equilibrium reached: Red on $S \to A \to T$ (cost 4), Blue on $S \to B \to C \to T$ (cost 3).

Neither player can improve $\Rightarrow$ **Nash Equilibrium**.

Question: Do Best Response Dynamics Always Converge?

No! Consider Matching Pennies:

	H	T
H	$1, -1$	$-1, 1$
T	$-1, 1$	$1, -1$

From any profile, some player has a profitable deviation $\Rightarrow$ **cycle forever**.

But: For **congestion games**, best-response dynamics *always* converges!

2.2.6 Rosenthal's Potential Function

Theorem (Rosenthal, 1973)

For every congestion game, every sequence of improvement steps is **finite**.

This property is called the **Finite Improvement Property (FIP)**.

Corollary

Every congestion game has at least one **pure strategy Nash equilibrium**.

Rosenthal's Potential Function

For any action profile a, define:

$$\boxed{\Phi(a) = \sum_{j=1}^{m} \sum_{k=1}^{n_j(a)} d_j(k)}$$

where $n_j(a)$ is the number of players using facility j under profile a.

Interpretation: Sum of "cumulative delay costs" over all facilities, summing delay values from 1 up to the actual congestion level.

Proof of Rosenthal's Theorem

Consider a single improvement step where player i switches from action a_i to b_i.

Change in player i's cost:

$$\Delta c_i = c_i(b_i, a_{-i}) - c_i(a_i, a_{-i}) = \sum_{j \in b_i \setminus a_i} d_j(n_j(a) + 1) - \sum_{j \in a_i \setminus b_i} d_j(n_j(a))$$

Since this is an improvement step: $\Delta c_i < 0$.

Change in potential:

$$\Delta\Phi = \Phi(b_i, a_{-i}) - \Phi(a_i, a_{-i}) = \sum_{j \in b_i \setminus a_i} d_j(n_j(a) + 1) - \sum_{j \in a_i \setminus b_i} d_j(n_j(a))$$

Key observation: $\Delta\Phi = \Delta c_i < 0$.

Since Φ takes finitely many values (finitely many profiles) and **strictly decreases** at each improvement step, the process must terminate. □

Exam Tip: The Potential Function Trick

The key insight is that the potential function exactly tracks individual cost changes:

- Facilities in $b_i \setminus a_i$: player joins, congestion increases by 1
- Facilities in $a_i \setminus b_i$: player leaves, congestion decreases by 1
- Facilities in $a_i \cap b_i$: no change to congestion

Both the player's cost change and the potential change depend only on the "boundary" facilities.

2.2.7 Potential Games

Exact Potential Game

A game $(N, (A_i)_{i \in N}, (U_i)_{i \in N})$ is an **exact potential game** if there exists a function $\Phi : A \to \mathbb{R}$ such that for every profile a, player i, and alternative action a_i':

$$U_i(a_i', a_{-i}) - U_i(a_i, a_{-i}) = \Phi(a_i', a_{-i}) - \Phi(a_i, a_{-i})$$

The function Φ is called the **exact potential function**.

Key insight: The change in any player's utility equals the change in the potential function.

Properties of Potential Games

- Every **exact potential game** has the Finite Improvement Property
- Every exact potential game has at least one **pure strategy Nash equilibrium**
- Local minima of Φ correspond to pure NE (for cost-minimisation games)
- **Congestion games are exact potential games** (Rosenthal's potential)

Exam Tip: Potential Games

To show a game is a potential game:

1. Propose a potential function Φ
2. Show that for any unilateral deviation, $\Delta U_i = \Delta \Phi$

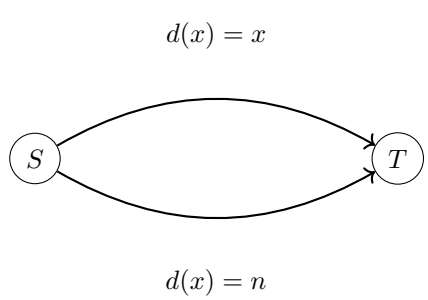
This immediately implies existence of pure NE and convergence of best-response dynamics.

2.2.8 Price of Anarchy and Price of Stability

Motivation: Inefficiency of Equilibria

Selfish behaviour can lead to socially suboptimal outcomes. We quantify this inefficiency by comparing equilibria to optimal solutions.

Pigou's Example



Setup: n players travel from S to T. Upper edge has cost = congestion; lower edge has constant cost n.

Selfish outcome: All players use the upper edge!

- If any player is on the lower edge (cost n), upper edge has $< n$ players, so cost $< n$
- Lower-edge player is "envious" and would switch
- **Equilibrium:** All on upper edge, each pays n, total cost $= n^2$

Social optimum: Split traffic equally

- $n/2$ on upper (each pays $n/2$), $n/2$ on lower (each pays n)
- Total cost $= \frac{n}{2} \cdot \frac{n}{2} + \frac{n}{2} \cdot n = \frac{n^2}{4} + \frac{n^2}{2} = \frac{3n^2}{4}$

Efficiency loss: $\dfrac{n^2}{3n^2/4} = \dfrac{4}{3}$

Price of Anarchy and Price of Stability

Given an optimisation objective (minimise cost or maximise welfare):

Price of Anarchy (PoA):

$$\text{PoA} = \frac{\text{value of worst Nash equilibrium}}{\text{value of optimal solution}}$$

Price of Stability (PoS):

$$\text{PoS} = \frac{\text{value of best Nash equilibrium}}{\text{value of optimal solution}}$$

Convention: For minimisation problems, PoA, PoS ≥ 1. For maximisation, invert the fraction so PoA, PoS ≥ 1.

Interpretation:

- PoA $= 1$: worst equilibrium is optimal (rare)
- PoS $= 1$: at least one optimal equilibrium exists
- Large PoA: selfish behaviour can be very costly

Prisoner's Dilemma: PoA and PoS

	C	D
C	$10, 10$	$0, 12$
D	$12, 0$	$1, 1$

Objective: Maximise social welfare (sum of utilities).

- **Unique NE:** (D, D) with welfare $1 + 1 = 2$
- **Social optimum:** (C, C) with welfare $10 + 10 = 20$

$$\text{PoA} = \text{PoS} = \frac{20}{2} = 10$$

Interpretation: Selfish behaviour loses a factor of 10 in efficiency!

Battle of the Sexes: PoA and PoS

	Theatre	Football
Theatre	$2, 1$	$0, 0$
Football	$0, 0$	$1, 2$

Objective: Maximise social welfare.

- **NE 1:** (Theatre, Theatre) with welfare $2 + 1 = 3$
- **NE 2:** (Football, Football) with welfare $1 + 2 = 4$
- **Social optimum:** welfare 4 (achieved by NE 2)

$$\text{PoS} = \frac{4}{4} = 1, \quad \text{PoA} = \frac{4}{3}$$

Interpretation: Best equilibrium is optimal, but coordination failure can cost $\frac{1}{3}$ of optimal welfare.

Three-Player Routing Game: PoA and PoS

Returning to our earlier example:

- **Unique NE:** State B (all use shared edge), social cost = 9
- **Social optimum:** State A, social cost = 8

$$\text{PoA} = \text{PoS} = \frac{9}{8}$$

2.2.9 Linear Congestion Games

Linear Congestion Game

A congestion game is **linear** if all cost functions have the form:

$$d_j(x) = a_j \cdot x + b_j$$

where $a_j \geq 0$ and $b_j \geq 0$ are constants.
Special case: If $b_j = 0$ for all j, the game has **identity cost functions**.

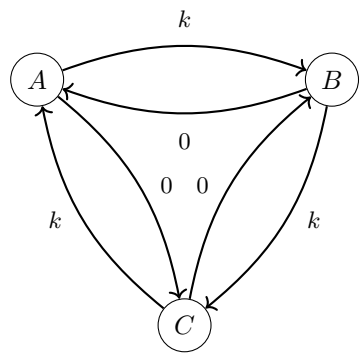
Theorem (Christodoulou and Koutsoupias, 2005)

For any congestion game with **linear cost functions**, the Price of Anarchy is at most **2.5**.

Theorem (Christodoulou and Koutsoupias, 2005)

There exist linear congestion games with 3 or more players where the Price of Anarchy equals **2.5** (tight bound).

Tight PoA = 2.5 Example (4 Players)



Players and destinations:

- Player 1: $A \to B$
- Player 2: $A \to C$
- Player 3: $B \to C$
- Player 4: $C \to B$

Edge costs: $d_j(x) = k \cdot x$ for "clockwise" edges; $d_j(x) = 0$ for "counter-clockwise" edges.

Optimal solution: Each player takes the direct path (cost k each).

- Total cost: $k + k + k + k = 4k$
- This is also a NE (any deviation uses at least one cost-k edge)

Worst NE: Players take longer paths:

- Player 1: $A \to C \to B$ (cost $0 + 2k = 2k$, but with congestion...)
- Player 2: $A \to B \to C$ (cost $2k + k$...)
- Player 3: $B \to A \to C$ (cost...)
- Player 4: $C \to A \to B$ (cost...)

After computing congestion effects:

- Players 1 and 2: cost $2k + k = 3k$ each (2 players on each edge they use)
- Players 3 and 4: cost $0 + 2k = 2k$ each
- Total: $3k + 3k + 2k + 2k = 10k$

Price of Anarchy: $\dfrac{10k}{4k} = \dfrac{10}{4} = 2.5$

2.2.10 Relating Potential Function to Social Cost

Lemma: Potential Bounds for Linear Congestion Games

For any linear congestion game and any strategy profile s:

$$\frac{1}{2} \cdot SC(s) \leq \Phi(s) \leq SC(s)$$

where $SC(s) = \sum_i c_i(s)$ is the social cost.

Proof Sketch

For linear costs $d_j(x) = a_j \cdot x + b_j$:

Social cost contribution from resource j:

$$SC_j = n_j(s) \cdot d_j(n_j(s)) = n_j(s) \cdot (a_j \cdot n_j(s) + b_j) = a_j \cdot n_j(s)^2 + b_j \cdot n_j(s)$$

Potential contribution from resource j:

$$\Phi_j = \sum_{k=1}^{n_j(s)} (a_j \cdot k + b_j) = a_j \cdot \frac{n_j(s)(n_j(s)+1)}{2} + b_j \cdot n_j(s)$$

Comparing:

- $\Phi_j \leq SC_j$ because $\frac{n(n+1)}{2} \leq n^2$ (equality when $n = 1$)
- $\Phi_j \geq \frac{1}{2} SC_j$ because $\frac{n(n+1)}{2} \geq \frac{n^2}{2}$

Exam Tip: Using Potential to Bound PoS

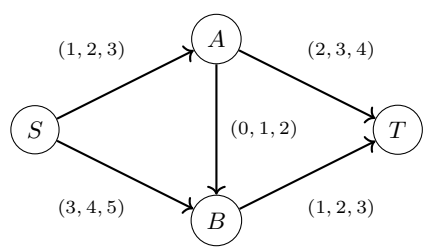
If a game has potential function Φ with $\alpha \cdot SC(s) \leq \Phi(s) \leq \beta \cdot SC(s)$, then:

$$\text{PoS} \leq \frac{\beta}{\alpha}$$

Because the best NE minimises Φ, and the optimal solution minimises SC.

2.2.11 Worked Examples from Exercises

Exercise 1: Finding NE in a Network Congestion Game



Setup: Three players, each travels $S \to T$. Edge labels (c_1, c_2, c_3) show cost when 1, 2, or 3 players use that edge.

Available paths:

1. $S \to A \to T$
2. $S \to B \to T$
3. $S \to A \to B \to T$

Solution approach:

1. List all strategy profiles (there are $3^3 = 27$, but symmetry helps)
2. For each profile, compute player costs
3. Check if any player can improve by switching

Candidate NE: All players take $S \to A \to T$.

- Edge (S, A): 3 players, cost = 3 each
- Edge (A, T): 3 players, cost = 4 each
- Total cost per player: $3 + 4 = 7$

Check deviations:

- Switch to $S \to B \to T$: cost $= 3 + 1 = 4 < 7$ ✓Profitable!

This is **not** a NE. Continue checking other profiles...

NE: (Found by systematic enumeration) Two players on $S \to A \to T$, one on $S \to B \to T$.

Exercise 2: Potential Function Bounds

Claim: For linear congestion games, $\frac{1}{2}SC(s) \leq \Phi(s) \leq SC(s)$.

Proof: For each resource j with n_j users and cost $d_j(x) = a_j x + b_j$:

Social cost contribution: $SC_j = n_j \cdot d_j(n_j) = a_j n_j^2 + b_j n_j$

Potential contribution:

$$\Phi_j = \sum_{k=1}^{n_j} (a_j k + b_j) = a_j \cdot \frac{n_j(n_j + 1)}{2} + b_j n_j$$

Upper bound: $\frac{n_j(n_j+1)}{2} \leq n_j^2$ (since $n_j + 1 \leq 2n_j$ for $n_j \geq 1$).

Lower bound: $\frac{n_j(n_j+1)}{2} = \frac{n_j^2}{2} + \frac{n_j}{2} \geq \frac{n_j^2}{2}$.

Summing over all resources gives the result. □

Exercise 3: PoS Bound Using Potential

Claim: If no resource is used by more than λ players, then PoS $\leq \lambda$.

Proof: Let s^* be the optimal profile and $\hat{s}$ be the best NE (minimises Φ).

Since NE minimises potential among all NE, and Φ strictly decreases along improvement paths:

$$\Phi(\hat{s}) \leq \Phi(s^*)$$

Using the bounds:

$$\frac{1}{\lambda} SC(\hat{s}) \leq \Phi(\hat{s}) \leq \Phi(s^*) \leq SC(s^*)$$

(The first inequality uses that with at most λ users, $\Phi_j \geq \frac{1}{\lambda} SC_j$.)

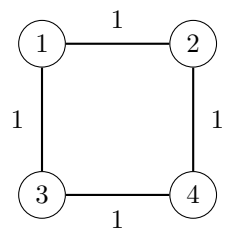
Therefore: $SC(\hat{s}) \leq \lambda \cdot SC(s^*)$, so PoS $\leq \lambda$. □

Exercise 4: Cut Games

Definition: Given graph $G = (V, E)$ with edge weights w_e:

- Each player i controls vertex v_i
- Strategy: place v_i in LEFT or RIGHT
- Utility: sum of weights of edges incident to v_i that cross the cut

(a) Design a cut game with PoA = 2:



Optimal: Diagonal cut (vertices 1,4 vs 2,3) cuts all 4 edges. Total utility = 8.

Worst NE: Horizontal cut (vertices 1,2 vs 3,4) cuts 2 edges. Total utility = 4.

PoA = $\frac{8}{4} = 2$.

(b) Prove PoA ≤ 2:

In any NE, each player must have at least half their adjacent edge weight in the cut (otherwise switching sides would be profitable).

For player i: $u_i \geq \frac{1}{2} \sum_{e \ni v_i} w_e$.

Summing: $\sum_i u_i \geq \frac{1}{2} \sum_i \sum_{e \ni v_i} w_e = \sum_e w_e$ (each edge counted twice).

Optimal welfare $\leq 2 \sum_e w_e$ (at most, each edge crosses the cut, counted for both endpoints).

Therefore: NE welfare $\geq \frac{1}{2}$ optimal $\Rightarrow$ PoA ≤ 2. □

2.2.12 Summary: Key Concepts

Concept	Key Points
Congestion Game	Players choose subsets of resources; cost depends on congestion only
Network Congestion Game	Resources are edges; actions are source-to-destination paths
Best Response Dynamics	Players iteratively improve; converges in congestion games
Rosenthal's Potential	$\Phi(a) = \sum_j \sum_{k=1}^{n_j(a)} d_j(k)$; decreases with each improvement
Finite Improvement Property	Every improvement sequence is finite $\Rightarrow$ pure NE exists
Price of Anarchy	Worst NE / Optimal (measures cost of selfishness)
Price of Stability	Best NE / Optimal (measures cost of decentralisation)
Linear Congestion Games	PoA ≤ 2.5; this bound is tight

3 Week 3: Repeated and Extensive-Form Games

3.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Define the core concepts of **extensive form games**, including game trees, choice nodes, and terminal nodes.
2. Formulate real-world scenarios as **extensive-form games** with perfect or imperfect information.
3. Identify **pure strategies** in extensive form games as complete plans specifying actions at every choice node.
4. Derive the **induced normal form** from an extensive form game and analyse its Nash equilibria.
5. Define and compute **subgame-perfect equilibria (SPE)** using backward induction.
6. Distinguish between **finitely** and **infinitely repeated games** and their equilibrium properties.
7. Analyse cooperation-sustaining strategies (**Tit-for-Tat**, **Trigger**) in infinitely repeated Prisoner's Dilemma.

The Big Picture

This week moves beyond normal form (simultaneous) games to settings where players act **sequentially** or **repeatedly**. Two key representations are introduced:

- **Extensive form games:** Game trees that model sequential moves, with **perfect information** (all history visible) or **imperfect information** (some moves hidden via information sets). The key refinement is **subgame-perfect equilibrium**, which eliminates non-credible threats.
- **Repeated games:** The same stage game played multiple times. Finitely repeated games collapse to stage-game NE by backward induction; **infinitely repeated games** can sustain cooperation via strategies like Tit-for-Tat and Trigger, provided the discount factor is sufficiently large.

What Examiners Like to Ask

- Write the **formal definition** of a perfect-information extensive form game (the 8-tuple).
- Given a game tree, enumerate all **pure strategies** for each player.
- Derive the **induced normal form** (payoff matrix) from a game tree.
- Find all **Nash equilibria** of the induced normal form, then identify which are **subgame-perfect**.
- Apply **backward induction** to compute SPE.
- Explain why every perfect-information game has a **pure strategy NE**.
- Show that the only NE of a **finitely repeated** Prisoner's Dilemma is always Defect.
- For infinitely repeated PD: compute the **minimum discount factor** β for Tit-for-Tat or Trigger to be a NE.
- Define **information sets** and explain the difference between perfect and imperfect information games.

3.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Extensive form games: motivation and applications
2. Perfect-information games: formal definition
3. Strategies in extensive form games
4. Induced normal form and its redundancy
5. Pure strategy NE existence theorem
6. Subgame-perfect equilibrium and backward induction
7. Imperfect-information games and information sets
8. Repeated games: finite and infinite
9. Payoff definitions: average reward and discounted reward
10. Cooperation strategies: Tit-for-Tat and Trigger

3.2.1 From Normal Form to Extensive Form

Why Extensive Form?

The normal form (payoff matrix) is universal but has limitations:

- Assumes **simultaneous** moves—cannot capture sequential structure
- Can be **exponentially larger** than the equivalent extensive form
- Misses solution concepts that depend on the **order of play** (e.g., subgame perfection)

Extensive form games are visualised as **game trees** and are widely used in AI:

Application	How Extensive Form Is Used
Chess, Go	Sequential moves modelled explicitly; minimax / alpha-beta pruning
Poker	Imperfect information; extensive-form fictitious self-play
Automated negotiation	Multi-stage strategic interaction with bluffing
Sequential auctions	Bids placed in turn; captures information and tactics
Security / patrolling	Sequential actions to maximise system utility (e.g., Stackelberg games)
Multi-agent RL	Sequential, stage-dependent strategies with private histories

---

3.2.2 Perfect-Information Extensive Form Games

Perfect-Information Game (Formal Definition)

A **finite perfect-information game** in extensive form is a tuple:

$$G = (N,\ A,\ H,\ Z,\ \chi,\ \rho,\ \sigma,\ u)$$

where:

- N: set of n players
- A: set of actions
- H: set of **nonterminal choice nodes**
- Z: set of **terminal nodes** (disjoint from H)
- $\chi : H \to 2^A$: **action function**—assigns available actions to each choice node
- $\rho : H \to N$: **player function**—assigns a player to each choice node
- $\sigma : H \times A \to H \cup Z$: **successor function**—maps (node, action) to the next node; injective (if $\sigma(h_1, a_1) = \sigma(h_2, a_2)$ then $h_1 = h_2$ and $a_1 = a_2$)
- $u = (u_1, \ldots, u_n)$: **utility functions** $u_i : Z \to \mathbb{R}$

Exam Tip: Tree Terminology

Since choice nodes form a tree:

- Each node can be identified with its **history**—the sequence of actions from the root
- The **descendants** of node h are all nodes in the subtree rooted at h

---

3.2.3 Strategies in Perfect-Information Games

Pure Strategy (Extensive Form)

A **pure strategy** for player i is a *complete* specification of which action to take at **every** choice node belonging to i:

$$s_i \in \prod_{h \in H,\ \rho(h)=i} \chi(h)$$

Key point: A strategy must specify an action at every node of player i, *even nodes that cannot be reached* given the other choices in the strategy.

Example: Sharing Game

Setup: A brother and sister share two identical presents. The brother proposes a split (2–0, 1–1, or 0–2), then the sister accepts or rejects. Rejection gives both 0.

Strategies:

- Player 1 (brother): 3 strategies: {2–0, 1–1, 0–2}
- Player 2 (sister): $2^3 = 8$ strategies (accept/reject at each of 3 nodes):
  $S_2 = \{(\text{Y,Y,Y}), (\text{Y,Y,N}), (\text{Y,N,Y}), \ldots, (\text{N,N,N})\}$

Example: Two-Player Game Tree with Induced Normal Form

Players: 1 has choice nodes with actions $\{A, B\}$ and $\{G, H\}$; 2 has choice nodes with actions $\{C, D\}$ and $\{E, F\}$.

Strategy sets: $S_1 = \{(A, G),\ (A, H),\ (B, G),\ (B, H)\}$, $\quad S_2 = \{(C, E),\ (C, F),\ (D, E),\ (D, F)\}$

Induced normal form:

	(C,E)	(C,F)	(D,E)	(D,F)
(A,G)	$3,8$	$3,8$	$8,3$	$8,3$
(A,H)	$3,8$	$3,8$	$8,3$	$8,3$
(B,G)	$5,5$	$2,10$	$5,5$	$2,10$
(B,H)	$5,5$	$1,0$	$5,5$	$1,0$

Redundancy: Only 5 distinct outcomes in the tree, but 16 cells in the matrix—many duplicated because unreachable nodes don't affect payoffs.

3.2.4 Extensive Form vs. Normal Form

Relationship Between Representations

- **Every** perfect-information extensive form game has a corresponding normal form game (the induced normal form).
- The reverse is **not** always possible—not every normal form game can be represented as a perfect-information extensive form game.
- **Example:** Prisoner's Dilemma cannot be represented as a perfect-information game (simultaneous moves require imperfect information).

Pure Strategy NE Existence

Every finite perfect-information game in extensive form has a **pure strategy Nash equilibrium**.

Proof idea: Since players take turns and observe all prior moves, randomisation is never needed—backward induction yields a pure-strategy equilibrium.

Important: This does **not** hold for imperfect-information games (e.g., Matching Pennies has no pure NE).

3.2.5 Subgame-Perfect Equilibrium

Subgame

Given a perfect-information game G, the **subgame** of G rooted at node h is the restriction of G to the descendants of h.

The set of subgames of G consists of all subgames rooted at any node in G (including G itself).

Subgame-Perfect Equilibrium (SPE)

A strategy profile s is a **subgame-perfect equilibrium** of G if, for every subgame G' of G, the restriction of s to G' is a Nash equilibrium of G'.

Key properties:

- Every SPE is a Nash equilibrium (but not vice versa)
- SPE eliminates equilibria based on **non-credible threats**
- Every perfect-information game has at least one SPE

Exam Tip: SPE vs NE

If a question asks "find all Nash equilibria, then identify which are subgame-perfect":

1. Find all NE from the induced normal form
2. Check each NE against every subgame
3. Discard any NE where a player's strategy is not a best response in some subgame (non-credible threat)

Example: Filtering NE to SPE

From the game tree example above, the induced normal form has three Nash equilibria:

- $\{(A, G),\ (C, F)\}, \quad \{(A, H),\ (C, F)\}, \quad \{(B, H),\ (C, E)\}$

Only $\{(B, H),\ (C, E)\}$ **is subgame-perfect**—the other two involve non-credible threats (actions at unreached nodes that are not best responses in their respective subgames).

3.2.6 Computing SPE: Backward Induction

Backward Induction Algorithm

Procedure:

1. Start at the **bottom-most** (deepest) subgames of the tree
2. At each terminal decision node, the player chooses the action maximising their payoff
3. Replace the subtree with the resulting payoff vector
4. Move up one level and repeat

Properties:

- Guarantees finding a **subgame-perfect** equilibrium
- Implemented as a single **depth-first traversal**—runs in time **linear** in the size of the game tree
- Avoids non-credible threats by construction

Centipede Game

In the Centipede game, backward induction predicts that the first player **immediately takes** (defects) at the first node, even though continued cooperation would yield higher payoffs. This is a classic case where the SPE prediction seems counterintuitive—in experiments, players often cooperate for several rounds.

3.2.7 Imperfect-Information Games

Imperfect-Information Game

A **finite imperfect-information game** extends the perfect-information tuple with **information sets**:

$$G = (N,\ A,\ H,\ Z,\ \chi,\ \rho,\ \sigma,\ u,\ I)$$

where:

- $(N, A, H, Z, \chi, \rho, \sigma, u)$ is a perfect-information game
- $I = (I_1, \ldots, I_n)$, where $I_i = (I_{i,1}, \ldots, I_{i,k_i})$ is a partition of player i's choice nodes such that for any two nodes h, h' in the same information set $I_{i,j}$:
  - $\chi(h) = \chi(h')$ (same available actions)
  - $\rho(h) = \rho(h')$ (same player)

Interpretation: If h and h' are in the same information set, the player **cannot distinguish** between them—they don't know which node they're at.

Pure Strategy (Imperfect Information)

A pure strategy for player i selects one action in each **information set** (not each node):

$$s_i \in \prod_{I_{i,j} \in I_i} \chi(I_{i,j})$$

Perfect-information games are the special case where every information set is a **singleton**.

Key Relationships

Direction	Result
Perfect info → normal form	Always possible (induced normal form)
Normal form → perfect info	**Not** always possible (e.g., Prisoner's Dilemma)
Normal form → imperfect info	Always possible (trivially: one information set per player)

Consequence: The set of perfect-information games is **strictly smaller** than the set of all normal form games. Imperfect-information games are equally expressive as normal form games.

Imperfect-information games may **not** always have a pure strategy NE (unlike perfect-information games).

3.2.8 Repeated Games

Repeated Game

A **repeated game** consists of the same **stage game** (in normal form) played multiple times by the same set of players.

Key questions:

- Do agents observe what others played in previous rounds?
- Do they remember their own history?
- How is the overall utility defined from the sequence of stage-game payoffs?

Strategies in Repeated Games

- A **stationary strategy**: play the same action in every round (memoryless)
- In general, the action at round t can depend on the **entire history** of play up to t
- History-dependent strategies are crucial for sustaining cooperation in infinitely repeated games

---

3.2.9 Finitely Repeated Games

Backward Induction in Finitely Repeated Games

In a finitely repeated game with a **known** number of rounds:

- Can be represented as an extensive form game with imperfect information
- Payoffs are typically **additive** (sum of stage-game payoffs)
- At each stage, players observe what was played in previous rounds but not the current round
- Backward induction applies: in the last round, play the stage-game NE; knowing this, play the stage-game NE in the second-to-last round; and so on

Result: The only NE of a finitely repeated Prisoner's Dilemma is to **always Defect** in every round.

Exam Tip: Finite Repetition Does Not Help

A common exam question: "Can cooperation be sustained in a finitely repeated Prisoner's Dilemma?" The answer is **no**—backward induction unravels cooperation from the last round.

3.2.10 Infinitely Repeated Games

Payoff Definitions for Infinite Games

Since the game never ends, payoffs cannot be attached to terminal nodes. Two standard definitions:

1. Average reward:

$$\bar{u}_i = \lim_{k\to\infty} \frac{\sum_{j=1}^{k} u_i^{(j)}}{k}$$

2. Future discounted reward (more common):

$$U_i = \sum_{j=1}^{\infty} \beta^j \cdot u_i^{(j)}$$

where $\beta \in [0, 1)$ is the **discount factor**.

Discount Factor β: Two Interpretations

1. **Time preference:** The agent cares more about near-term payoffs than distant ones.
2. **Continuation probability:** The game continues to the next round with probability β; it stops with probability $1 - \beta$.

A higher β (closer to 1) means the agent is more **patient** / the game is more likely to continue.

3.2.11 Sustaining Cooperation: Tit-for-Tat and Trigger

Tit-for-Tat (TFT)

1. Start by **cooperating**
2. In round $j + 1$, play whatever the opponent played in round j

Properties: Forgiving (returns to cooperation after opponent cooperates again), simple, retaliatory.

Trigger Strategy (Grim Trigger)

1. Start by **cooperating**
2. If the opponent **ever** defects, defect **forever**

Properties: Unforgiving (permanent punishment), strong deterrent, harsher than TFT.

Cooperation in Infinitely Repeated PD

Both Tit-for-Tat and Trigger strategy are **Nash equilibria** in the infinitely repeated Prisoner's Dilemma for **sufficiently large** discount factor β.

Moreover, they form an equilibrium not just with themselves but also **with each other**—if one player uses TFT and the other uses Trigger, mutual cooperation is sustained.

Intuition: The short-term gain from defecting is outweighed by the long-run punishment, provided the player cares enough about future payoffs (β large enough).

Computing Minimum β for Trigger Strategy

Consider the Prisoner's Dilemma with payoffs:

	C	D
C	$-1, -1$	$-4, 0$
D	$0, -4$	$-3, -3$

If both play Trigger (cooperate forever):

$$U_{\text{coop}} = \sum_{j=1}^{\infty} \beta^j(-1) = \frac{-\beta}{1-\beta}$$

If player deviates (defect once, then both defect forever):

$$U_{\text{dev}} = \beta \cdot 0 + \sum_{j=2}^{\infty} \beta^j(-3) = \frac{-3\beta^2}{1-\beta}$$

Cooperation is a NE when: $U_{\text{coop}} \geq U_{\text{dev}}$

$$\frac{-\beta}{1-\beta} \geq \frac{-3\beta^2}{1-\beta} \implies -1 \geq -3\beta \implies \beta \geq \frac{1}{3}$$

So Trigger sustains cooperation for $\beta \geq \frac{1}{3}$.

3.2.12 Summary: Key Concepts

Concept	Key Fact
Perfect-info extensive form	8-tuple $(N, A, H, Z, \chi, \rho, \sigma, u)$; game tree with full history visible
Pure strategy (extensive)	Complete plan: one action per choice node (or info set), even unreachable ones
Induced normal form	Exponentially larger; many cells map to the same leaf
Perfect info $\Rightarrow$ pure NE	Always exists (backward induction)
SPE	NE in every subgame; eliminates non-credible threats
Backward induction	Depth-first, linear-time algorithm to compute SPE
Imperfect info	Information sets partition choice nodes; player cannot distinguish nodes in same set
Finitely repeated PD	Only NE is always Defect (backward induction unravels)
Infinitely repeated PD	Cooperation via TFT/Trigger for large enough β
Discount factor β	Time preference or continuation probability; higher β = more patient

4 Week 4: Cooperative Games (Part I)

4.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Explain the fundamental concepts of cooperative game theory, including **coalitions**, **transferable utility**, and the difference from non-cooperative settings.
2. Model real-world scenarios as cooperative games, identifying possible coalitions and collective payoff structures.
3. Analyse solution concepts such as the **core**, **Shapley value**, and **Banzhaf index**, including their properties and interpretation.
4. Determine when stable and fair coalition structures exist, and evaluate conditions for allocations to be in the core.
5. Calculate the Shapley value and Banzhaf index using mathematical reasoning.
6. Assess the fairness, efficiency, and stability of division outcomes in cooperative environments.

The Big Picture

This week shifts from non-cooperative settings (where players act individually) to **cooperative game theory**, where players can form **binding agreements** and work in **coalitions**. The key premise: cooperation is not altruism—it emerges because players find it mutually beneficial to commit to agreements.

Two central questions drive the theory:

- **Stability:** Which outcomes are immune to coalitional deviations? → **The Core**
- **Fairness:** How should payoffs reflect each player's contribution? → **Shapley Value**, **Banzhaf Index**

What Examiners Like to Ask

- Write the **formal definition** of a TU game (N, v) and explain the characteristic function.
- Given a characteristic function, determine whether the game is **superadditive**, **convex**, or **simple**.
- Compute the **core** of a game by writing down the constraint system and finding feasible payoff vectors.
- Show that a given game has an **empty core** (e.g., 3-player majority game).
- Prove that **convex games** have non-empty cores.
- Prove that a **simple game** has a non-empty core iff it has a veto player.
- Compute the **Shapley value** for all players (enumerate permutations, compute marginal contributions).
- Compute the **Banzhaf index** and compare with the Shapley value.
- State and apply the **four axiomatic properties** of the Shapley value.
- Identify **null players** and **veto players** in simple/weighted voting games.

4.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Cooperative vs. non-cooperative games; binding agreements
2. TU vs. NTU games; characteristic vs. partition function games
3. Formal definition of TU games: (N, v)
4. Outcomes, coalition structures, and imputations
5. Special classes: simple, superadditive, convex games
6. The Core: definition, examples, emptiness
7. Core of convex games and simple games
8. Relaxations: ε-core, least core, cost of stability
9. Shapley value: definition, computation, properties
10. Banzhaf index

4.2.1 From Non-Cooperative to Cooperative Games

Cooperative (Coalitional) Game

In **cooperative games**:

- Agents can benefit by cooperating
- **Binding agreements** are possible (unlike Prisoner's Dilemma)
- Actions are taken by **coalitions** of agents
- Focus shifts from individual strategies to *which coalitions form* and *how payoffs are divided*

TU vs. NTU Games

	TU (Transferable Utility)	**NTU (Non-Transferable Utility)**
Payoff	Given to the group, then divided	Directly to individual members
Side payments	Feasible (money, goods)	Not possible
Example	Farmers splitting profit	Researchers getting promotions
Key question	How to split the surplus?	Which coalitions form?

CFG vs. PFG

- **Characteristic Function Game (CFG):** A coalition's payoff depends *only* on its own composition. Each coalition is identified with a single number $v(C)$.
- **Partition Function Game (PFG):** A coalition's payoff depends on the *entire partition*—what other coalitions are doing matters (e.g., market competition).

Most classical cooperative game theory (core, Shapley value) was developed for CFGs.

4.2.2 Transferable Utility Games: Formal Definition

TU Game

A **transferable utility game** is a pair (N, v) where:

- $N = \{1, \ldots, n\}$ is a set of **players**
- $v : 2^N \to \mathbb{R}$ is the **characteristic function**
- For each coalition $C \subseteq N$, $v(C)$ is the total payoff the members of C can earn together

Standard assumptions:

- **Normalised:** $v(\emptyset) = 0$
- **Non-negative:** $v(C) \geq 0$ for all $C \subseteq N$
- **Monotone:** $C \subseteq D \implies v(C) \leq v(D)$

N itself is called the **grand coalition**.

Outcome of a TU Game

An **outcome** of (N, v) is a pair $(\mathrm{CS}, \mathbf{x})$ where:

- $\mathrm{CS} = (C_1, \ldots, C_k)$ is a **coalition structure**—a partition of N
- $\mathbf{x} = (x_1, \ldots, x_n)$ is a **payoff vector** satisfying:
  - $x_i \geq 0$ for all $i \in N$
  - $x(C) = \sum_{i \in C} x_i = v(C)$ for each C in CS

An outcome is an **imputation** if it also satisfies **individual rationality**: $x_i \geq v(\{i\})$ for all i.

Running Example: Ice-Cream Game

Three children: Charlie (£4), Marcy (£3), Pattie (£2). Ice-cream tubs: 1kg for £9, 750g for £7, 500g for £5. Children value ice-cream, not money.

Characteristic function:

Coalition	$\emptyset$	$\{C\}$	$\{M\}$	$\{P\}$	$\{C, P\}$	$\{M, P\}$	$\{C, M\}$
$v(C)$	0	0	0	0	500	500	750

$v(\{C, M, P\}) = 1000$

Some outcomes:

- $\mathrm{CS} = \{\{C\}, \{M, P\}\}$; $x = (0, 150, 350)$
- $\mathrm{CS} = \{N\}$; $x = (334, 333, 333)$
- $\mathrm{CS} = \{N\}$; $x = (0, 800, 200)$ (feasible but arguably unfair)

4.2.3 Important Game Types

Weighted Voting Games (WVG)

Players have weights $w_1, \ldots, w_n$ and a quota q:

$$v(C) = \begin{cases} 1 & \text{if } w(C) = \sum_{i \in C} w_i \geq q \quad \text{(winning)} \\ 0 & \text{otherwise} \quad \text{(losing)} \end{cases}$$

Notation: WVG$[q;\ w_1, w_2, \ldots, w_n]$

UK Elections as WVGs

Year	Conservatives	Veto player?	Null players?
2010	306 seats	None	None
2019	365 seats	C (veto)	All others (null)

(Quota = 326 in all cases, 650 total seats)

Other Game Types

- **Network flow games:** Players are edges; $v(C) = \max\ s$–t flow through edges in C
- **Graph-induced games:** Players are vertices; $v(C)$ = total weight of internal edges (can be negative)

4.2.4 Special Classes of TU Games

Simple Games

A game (N, v) is **simple** if:

- $v(C) \in \{0, 1\}$ for all $C \subseteq N$
- v is monotone

Coalitions are **winning** ($v = 1$) or **losing** ($v = 0$).

Convention: In simple games, assume the grand coalition forms (outcome = payoff vector).

Null and Veto Players (Simple Games)

- Player i is a **null player** if: $v(C \cup \{i\}) = v(C)$ for all $C \subseteq N \setminus \{i\}$
  (Adding i never changes any coalition's value)
- Player i is a **veto player** if: $v(C) = 0$ for all $C \subseteq N \setminus \{i\}$
  (Equivalently, $v(N \setminus \{i\}) = 0$—without i, no coalition can win)

Superadditive Games

A game (N, v) is **superadditive** if:

$$v(S \cup T) \geq v(S) + v(T) \quad \text{for any disjoint } S, T \subseteq N$$

Interpretation: Two coalitions can always merge without losing value $\Rightarrow$ grand coalition always forms.

Convention: In superadditive games, outcomes are identified with payoff vectors $\mathbf{x}$ satisfying $x(N) = v(N)$.

Convex Games (Supermodularity)

A function $f : 2^N \to \mathbb{R}$ is **supermodular** if $f(\emptyset) = 0$ and:

$$f(A \cup B) + f(A \cap B) \geq f(A) + f(B) \quad \text{for all } A, B \subseteq N$$

A game (N, v) is **convex** if v is supermodular.

Key property (increasing marginal contributions): If $T \subset S$ and $i \notin S$, then:

$$v(T \cup \{i\}) - v(T) \leq v(S \cup \{i\}) - v(S)$$

A player is **more useful** when joining a **bigger** coalition.

Hierarchy: Convex $\Rightarrow$ Superadditive (but not vice versa).

Properties of Game Types

Game Type	Superadditive?	Convex?
Ice-cream	Yes	No
Graph-induced	Iff all edge weights ≥ 0	Iff all edge weights ≥ 0
Network flow	Yes	Not necessarily
WVG (general)	Not always	Not always
WVG with $q > w(N)/2$	Yes	Not necessarily

WVG counterexample for superadditivity: $N = \{1, 2\}$, $w_1 = w_2 = 1$, $q = 1$. Each singleton wins alone, so $v(\{1\}) + v(\{2\}) = 2 > 1 = v(N)$.

WVG counterexample for convexity: $N = \{1, 2, 3\}$, $w_i = 1$, $q = 2$. Then $v(\{1, 2\}) + v(\{2, 3\}) = 2 > 1 = v(\{1, 2, 3\}) + v(\{2\})$.

4.2.5 The Core

The Core

The **core** $\mathcal{C}(G)$ of a characteristic function game $G = (N, v)$ is the set of all outcomes $(\text{CS}, \mathbf{x})$ such that:

$$x(C) \geq v(C) \quad \text{for every } C \subseteq N$$

Interpretation: No coalition can deviate and earn more on its own—every coalition gets at least what it could guarantee itself.

For **superadditive** games, the core simplifies to all payoff vectors $\mathbf{x}$ with:

- $x_i \geq 0$ for all i
- $x(N) = v(N)$
- $x(C) \geq v(C)$ for all $C \subseteq N$

Core Stability Implies Efficiency

If $(\text{CS}, \mathbf{x})$ is in the core, then CS maximises social welfare:

$$v(\text{CS}) \geq v(\text{CS}') \quad \text{for every coalition structure CS}' \text{ over } N$$

Proof sketch: Suppose $v(\text{CS}) < v(\text{CS}')$. Then $\sum_{C' \in \text{CS}'} x(C') = v(\text{CS}) < v(\text{CS}') = \sum_{C' \in \text{CS}'} v(C')$. But core stability requires $x(C') \geq v(C')$ for each C', giving $\sum x(C') \geq \sum v(C')$—contradiction.

Core of the Ice-Cream Game

Since the game is superadditive, $\text{CS} = \{N\}$. Core constraints:

$$\begin{aligned} x_C \geq 0,\ x_M \geq 0,\ x_P \geq 0 \\ x_C + x_M + x_P &= 1000 \\ x_C + x_M &\geq 750 \\ x_C + x_P &\geq 500 \\ x_M + x_P &\geq 500 \end{aligned}$$

Solutions include: $(400, 400, 200)$, $(500, 400, 100)$, $(500, 500, 0)$.

Note: $(350, 350, 300)$ is **not** in the core because $x_C + x_M = 700 < 750$.

Empty Core: 3-Player Majority Game

WVG[2; 1, 1, 1]: $v(C) = 1$ if $|C| \geq 2$; $v(C) = 0$ if $|C| \leq 1$.

The core is empty:

- If $\text{CS} = \{\{1\}, \{2\}, \{3\}\}$: grand coalition deviates (all get 0, but $v(N) = 1$)
- If $\text{CS} = \{\{1, 2\}, \{3\}\}$: one of $\{1, 2\}$ gets < 1, can deviate with player 3
- If $\text{CS} = \{N\}$: some $x_i > 0$, so $x(N \setminus \{i\}) < 1 = v(N \setminus \{i\})$

No outcome satisfies all core constraints simultaneously.

4.2.6 Core of Special Game Classes

Core of Convex Games

Every **convex game** has a non-empty core.

Constructive proof: Order players as $1, \ldots, n$. Set:

$$x_i = v(\{1, \ldots, i\}) - v(\{1, \ldots, i-1\})$$

(Pay each player their marginal contribution to the coalition of predecessors.)

Then $\sum x_i = v(N)$ (telescoping). For any $C = \{i, j, \ldots, s\}$ with $i < j < \ldots < s$:

$$v(C) = \sum_{\text{members}} [\text{marginal to predecessors in } C] \leq \sum_{\text{members}} x_k$$

by supermodularity (marginal contribution to a subset $\leq$ marginal contribution to a superset).

Complexity: A core outcome can be constructed in $O(n)$ queries to v.

Core of Simple Games

A simple game has a non-empty core **if and only if** it has a **veto player**.

Proof ($\Leftarrow$): If i is a veto player, set $x_i = 1$, $x_k = 0$ for $k \neq i$. For any C: if $i \in C$, then $x(C) = 1 \geq v(C)$; if $i \notin C$, then $v(C) = 0$.

Proof ($\Rightarrow$): Suppose no veto player. Take any payoff vector $\mathbf{x}$ with $x(N) = 1$. Then some $x_i > 0$, so $x(N \setminus \{i\}) < 1$. But since i is not a veto player, $v(N \setminus \{i\}) = 1$—core constraint violated.

Corollary: Core Characterisation for Simple Games

In a simple game, $\mathbf{x}$ is in the core iff $x_i = 0$ for every non-veto player i.

Checking core non-emptiness requires a single query: compute $v(N \setminus \{i\})$ for each i.

Exam Tip: WVG with Coalition Structures

If coalition structures (not just grand coalition) are allowed in WVGs, deciding core non-emptiness becomes **NP-hard** (reduction from PARTITION).

Example: WVG[2; 1, 1, 1, 1]—the outcome $(\{\{1,2\},\{3,4\}\}, (\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}))$ is in the core, even though there is no veto player.

4.2.7 Core Relaxations

ε-Core

An outcome $\mathbf{x}$ is in the **ε-core** of a superadditive game G (for $\varepsilon \geq 0$) if:

$$x(C) \geq v(C) - \varepsilon \quad \text{for all } C \subseteq N$$

Allows deviations only if the gain exceeds ε (accounts for deviation costs).

Least Core

The **least core** of G is the ε^*-core where:

$$\varepsilon^*(G) = \inf\{\varepsilon \mid \varepsilon\text{-core of } G \text{ is non-empty}\}$$

Can be computed via LP:

$$\min \varepsilon \quad \text{s.t.} \quad x_i \geq 0,\ x(N) = v(N),\ x(C) \geq v(C) - \varepsilon\ \forall C \subseteq N$$

The least core is **always non-empty**.

Cost of Stability

The **cost of stability** of $G = (N, v)$ is the smallest $\Delta \geq 0$ such that the game $G^\Delta = (N, v^\Delta)$ with $v^\Delta(N) = v(N) + \Delta$ and $v^\Delta(C) = v(C)$ for $C \subsetneq N$ has a non-empty core.

Computed via LP: $\min \Delta$ s.t. $x_i \geq 0$, $x(N) = v(N) + \Delta$, $x(C) \geq v(C)$ for all $C \subseteq N$.

4.2.8 Shapley Value

Shapley Value

Given a game $G = (N, v)$ with $|N| = n$, the **Shapley value** of player i is:

$$\phi_i(G) = \frac{1}{n!} \sum_{\pi \in \Pi_N} \Delta_G^\pi(i)$$

where Π_N is the set of all permutations of N, and the **marginal contribution** of i w.r.t. permutation π is:

$$\Delta_G^\pi(i) = v(S_\pi(i) \cup \{i\}) - v(S_\pi(i))$$

with $S_\pi(i) = \{j \in N \mid \pi(j) < \pi(i)\}$ (predecessors of i in π).

Probabilistic interpretation: ϕ_i is i's expected marginal contribution when players arrive in a uniformly random order.

Shapley Value: Ice-Cream Game (Pattie)

Six permutations of $\{C, M, P\}$:

Permutation	**Predecessors of P**	$\Delta^\pi(P)$
CMP	$\{C, M\}$	$v(N) - v(\{C, M\}) = 250$
MCP	$\{M, C\}$	$v(N) - v(\{C, M\}) = 250$
CPM	$\{C\}$	$v(\{C, P\}) - v(\{C\}) = 500$
MPC	$\{M\}$	$v(\{M, P\}) - v(\{M\}) = 500$
PCM	$\emptyset$	$v(\{P\}) - v(\emptyset) = 0$
PMC	$\emptyset$	$v(\{P\}) - v(\emptyset) = 0$

$\phi_P = \frac{250+250+500+500+0+0}{6} = 250$

Shapley Value Using Symmetry

Game: $v(N) = 1$, $v(C) = 0$ for all $C \subsetneq N$, with $|N| = n$.

Method 1 (direct): $\Delta^\pi(i) = 1$ only if i is last in π. There are $(n-1)!$ such permutations. $\phi_i = \frac{(n-1)!}{n!} = \frac{1}{n}$

Method 2 (properties): All players are symmetric $\Rightarrow$ equal Shapley values. By efficiency: $n \cdot \phi_i = v(N) = 1 \Rightarrow \phi_i = \frac{1}{n}$.

4.2.9 Shapley Value: Axiomatic Properties

The Four Properties of the Shapley Value

The Shapley value is the **unique** payoff division scheme satisfying:

#	**Property**	**Statement**
1	**Efficiency**	$\sum_{i \in N} \phi_i(G) = v(N)$
2	**Null (Dummy) player**	If $v(C \cup \{i\}) = v(C)$ for all C, then $\phi_i(G) = 0$
3	**Symmetry**	If $v(C \cup \{i\}) = v(C \cup \{j\})$ for all $C \subseteq N \setminus \{i, j\}$, then $\phi_i = \phi_j$
4	**Additivity**	$\phi_i(G_1 + G_2) = \phi_i(G_1) + \phi_i(G_2)$

where $(G_1 + G_2) = (N, v^+)$ with $v^+(C) = v^1(C) + v^2(C)$.

Exam Tip: Using Properties to Compute Shapley Value

The four properties can simplify computation:

1. **Symmetry** reduces distinct computations (symmetric players get equal shares)
2. **Null player** eliminates players with zero contribution

3. **Efficiency** pins down the total
4. **Additivity** lets you decompose complex games into simpler ones

---

4.2.10 Banzhaf Index

Banzhaf Index

Given $G = (N, v)$ with $|N| = n$, the **Banzhaf index** of player i is:

$$\beta_i(G) = \frac{1}{2^{n-1}} \sum_{C \subseteq N \setminus \{i\}} [v(C \cup \{i\}) - v(C)]$$

Key difference from Shapley: Averages over all **coalitions** (of which there are 2^{n-1} not containing i), rather than all permutations ($n!$).

Banzhaf vs. Shapley: Property Comparison

Property	**Shapley**	**Banzhaf**
Efficiency	✓	×
Null (Dummy) player	✓	✓
Symmetry	✓	✓
Additivity	✓	✓

The Banzhaf index does **not** redistribute $v(N)$—the sum $\sum \beta_i$ may differ from $v(N)$.

Banzhaf Index Example

Game: $v(N) = 1$, $v(C) = 0$ for $C \subsetneq N$, $|N| = n$.

For player i: only $C = N \setminus \{i\}$ gives marginal contribution 1; all other $2^{n-1} - 1$ coalitions give 0.

$\beta_i = \frac{1}{2^{n-1}}$

$\sum_{i \in N} \beta_i = \frac{n}{2^{n-1}}$, which for $n > 2$ is $< 1 = v(N)$.

Normalised Banzhaf Index

To ensure efficiency:

$$\nu_i(G) = \frac{\beta_i(G)}{\sum_{j \in N} \beta_j(G)}$$

This fixes efficiency but **breaks additivity**.

Shapley-Shubik Power Index

In **simple games** (where $v(C) \in \{0, 1\}$), the Shapley value and Banzhaf index measure the **power** of a player—the probability that the player can influence the outcome. The Shapley value in this context is called the **Shapley-Shubik power index**.

4.2.11 Summary: Key Concepts

Concept	**Key Fact**
TU game (N, v)	Characteristic function $v : 2^N \to \mathbb{R}$; normalised, non-negative, monotone
Coalition structure	Partition of N; payoff vector distributes each coalition's value
Superadditive	Merging never loses value; grand coalition always forms
Convex	Supermodular v; increasing marginal contributions; always has non-empty core
Simple game	$v(C) \in \{0, 1\}$; non-empty core iff veto player exists
Core	Stable outcomes: no coalition can deviate; implies efficiency
ε-core / least core	Relaxation allowing bounded instability; always non-empty
Shapley value	Unique: efficient + null player + symmetry + additivity
Banzhaf index	Averages over coalitions (not permutations); not efficient

5 Week 5: Cooperative Games (Part II)

5.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Describe the core concepts of **hedonic games**, including non-transferable utility, preferences over coalitions, and compact representations.
2. Model coalition formation problems using **additively separable hedonic games** (ASHGs) and related classes.
3. Analyse stability concepts for hedonic games: **individual rationality**, **Nash stability**, **individual stability**, **contractual individual stability**, and **core stability**.
4. Describe the **stable matching problem**, including preference profiles, stability, and blocking pairs.
5. Apply the **Gale-Shapley deferred acceptance algorithm** to find stable matchings in bipartite settings.
6. Prove key properties of DA: termination, perfection, stability, man-optimality, women-pessimality.
7. Evaluate fairness and strategic aspects of stable matchings, including strategyproofness.
8. Interpret applications of stable matching theory (college admissions, medical residencies, school choice).

The Big Picture

Week 4 covered **transferable utility** (TU) games where coalitions split a shared surplus. This week moves to **non-transferable utility** (NTU) settings where payoffs accrue directly to individuals and cannot be redistributed. The central model is the **hedonic game**, where each player's utility depends only on the composition of their coalition.

The second half introduces the **stable matching problem**—a special hedonic game where agents from two sides (e.g. men/women, students/hospitals) must be paired. The **Gale-Shapley algorithm** always finds a stable matching, and its properties (optimality, pessimality, strategyproofness) have profound practical consequences.

What Examiners Like to Ask

- Given an ASHG with a parameter, determine for which values a coalition structure is **individually rational**, **Nash stable**, or **in the core**.
- State the hierarchy of stability concepts and give examples where one holds but a stronger one fails.
- Trace the **Gale-Shapley algorithm** on a given preference instance step-by-step.
- Construct worst-case instances where DA requires many proposals.
- Explain why DA is **man-optimal** and **women-pessimal**; reproduce the proof.
- Determine whether DA is **strategyproof** for men, women, or both.
- Give an instance of the **stable roommate problem** with no stable matching.
- Describe extensions: many-to-one matching, unacceptable pairs, indifferences.

5.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Non-transferable utility (NTU) games and hedonic games
2. Compact representations: ASHGs, fractional, B-games, W-games
3. Stability concepts: IR, Nash, individual, CIS, core
4. Friends-and-enemies games
5. Stable matching problem: formal model
6. Gale-Shapley deferred acceptance algorithm
7. Properties: termination, perfection, stability, optimality
8. Strategyproofness
9. Extensions and applications

5.2.1 Non-Transferable Utility Games

NTU Games

In **non-transferable utility games**:

- Payoffs accrue to **individual** coalition members and **cannot be transferred**.
- A coalition may have **multiple actions** producing different payoff vectors for its members.
- Example: coalition $\{x, y\}$ can perform action a (payoffs $1, 6$) or action b (payoffs $4, 2$).

Motivating example: n researchers at different universities form groups to write papers. Each author receives rewards from their own university (promotion, bonus, teaching reduction)—these are **non-transferable**.

5.2.2 Hedonic Games

Hedonic Game

A **hedonic game** is a pair $(N, (\succeq_i)_{i \in N})$ where:

- $N = \{1, \ldots, n\}$ is a set of **players**
- Each player i has a preference relation $\succeq_i$ over $\{S \subseteq N \mid i \in S\}$ (all coalitions containing i)
- The outcome is a **coalition structure** (partition of N)—no payoff distribution phase
- Player i's utility depends **only** on the coalition i belongs to

Special case of NTU games: each coalition has only **one action** available (forming itself).

Hedonic Game Example

Three players with preferences:

- 1: $\{1, 2\} \sim \{1, 3\} \succ \{1, 2, 3\} \succ \{1\}$
- 2: $\{2\} \succ \{1, 2, 3\} \succ \{1, 2\} \succ \{2, 3\}$
- 3: $\{1, 2, 3\} \succ \{3\} \succ \{2, 3\} \sim \{1, 3\}$

Preferences can be encoded numerically: e.g. $v_1(\{1, 2\}) = v_1(\{1, 3\}) = 2$, $v_1(\{1, 2, 3\}) = 1$, $v_1(\{1\}) = 0$.

5.2.3 Compactly Representable Hedonic Games

Full hedonic preferences are exponential in n. Several compact representations exist:

Compact Representations

Each agent i assigns a value $v_i(j)$ to every other agent $j \in N \setminus \{i\}$:

Class	**Utility of i in coalition S**
Additively Separable (ASHG)	$v_i(S) = \sum_{j \in S \setminus \{i\}} v_i(j)$
Fractional	$v_i(S) = \frac{\sum_{j \in S \setminus \{i\}} v_i(j)}{\lvert S \rvert}$
B-game (Best)	$v_i(S) = \max_{j \in S \setminus \{i\}} v_i(j)$
W-game (Worst)	$v_i(S) = \min_{j \in S \setminus \{i\}} v_i(j)$

Also: **anonymous games** where preferences depend on coalition size only.

ASHGs vs. Induced Subgraph Games

Both use a weighted graph representation (players = vertices):

	Induced Subgraph Games	**ASHGs**
Edges	Undirected	Directed
Value of coalition T	Total weight of internal edges	Sum of i's outgoing edges to $T \setminus \{i\}$
Symmetry	Always symmetric	Not necessarily

5.2.4 Stability Concepts for Hedonic Games

The stability concepts form a hierarchy (strongest to weakest):

$$\text{Core stable} \Rightarrow \text{Nash stable} \Rightarrow \text{Individually stable} \Rightarrow \text{CIS} \Leftarrow \text{(always exists)}$$

Also: Nash stable $\Rightarrow$ Individually rational.

Individual Rationality (IR)

A partition is **individually rational** if no player strictly prefers being alone:

$$v_i(S_i) \geq v_i(\{i\}) \quad \text{for all } i \in N$$

where S_i is i's coalition in the partition. This is the **minimum stability requirement**.

Nash Stability (NS)

Player i has an **NS-deviation** from partition P if there exists $T \in P \cup \{\emptyset\}$ such that i prefers $T \cup \{i\}$ to their current coalition.

A partition is **Nash stable** if no player has an NS-deviation.

Key: Player i can move unilaterally to any existing coalition (or form a singleton)—**no consent needed** from the receiving coalition.

Nash Stability May Not Exist

There exists a two-player game with no Nash stable outcome: if the red player values the yellow player positively but yellow values red negatively, then $\{\{r, y\}\}$ is unstable (yellow deviates) and $\{\{r\}, \{y\}\}$ is unstable (red deviates).

Nash Stability in Symmetric ASHGs

In **symmetric ASHGs** ($v_i(j) = v_j(i)$ for all i, j):

- Every such game admits a Nash stable outcome.
- Every NS-deviation increases the sum of utilities (social welfare).
- Finding a Nash stable outcome is **PLS-complete** (Gairing, Savani '10).

Individual Stability (IS)

Player i has an **IS-deviation** from partition P if there exists $T \in P \cup \{\emptyset\}$ such that:

1. i prefers $T \cup \{i\}$ to their current coalition, **and**
2. all players in T **weakly prefer** $T \cup \{i\}$ to T.

Compared to NS: the receiving coalition must **approve** the newcomer.

Contractual Individual Stability (CIS)

Player i has a **CIS-deviation** from partition P if there exists $T \in P \cup \{\emptyset\}$ such that:

1. i prefers $T \cup \{i\}$ to their current coalition S
2. All players in T weakly prefer $T \cup \{i\}$ to T (receiving coalition approves)
3. All players in S weakly prefer $S \setminus \{i\}$ to S (leaving coalition approves)

Every Hedonic Game Has a CIS Outcome

Proof:

1. Encode preferences numerically (always possible).
2. Every CIS-deviation **increases** the sum of utilities across all agents.
3. A partition that **maximises** the sum of utilities must be CIS (no improving deviation exists). □

Core Stability

A group $C \subseteq N$ **blocks** a partition P if every player in C prefers C to their coalition in P.

A partition is **core stable** if no group blocks it.

Empty Core in a Hedonic Game

Three players:

- 1: $\{1, 2\} \succ \{1, 3\} \succ \{1\} \succ \{1, 2, 3\}$
- 2: $\{2, 3\} \succ \{2, 1\} \succ \{2\} \succ \{1, 2, 3\}$
- 3: $\{3, 1\} \succ \{3, 2\} \succ \{3\} \succ \{1, 2, 3\}$

Analysis: Grand coalition is blocked (any singleton prefers to leave). Singletons are blocked (any pair prefers to form). Any pair $\{i, j\}$ is blocked: the non-favourite member of the pair prefers to leave and join the singleton. The core is **empty**.

Core of ASHGs

- The core may be empty even for **symmetric** ASHGs (5-player counterexample with negative weights: no coalition of size ≥ 3 is IR; no partition into singletons and pairs is stable).
- If all edge weights are **non-negative**, the grand coalition is core stable (trivially: every agent's utility is maximised).

5.2.5 Friends-and-Enemies Games

Friends and Enemies Games

A subclass of ASHGs where each player i partitions $N \setminus \{i\}$ into **friends** F_i and **enemies** E_i:

- Utility $F > 0$ for each friend, utility $E < 0$ for each enemy.
- Need **not** be symmetric (i may consider j a friend while j considers i an enemy).

Two special cases with guaranteed non-empty cores:

Game	Friend utility	Enemy utility	Intuition
Friend appreciation	$F = n$	$E = -1$	Friends matter a lot
Enemy aversion	$F = 1$	$E = -n$	Enemies are very costly

Core of Friends-and-Enemies Games

Every **friend appreciation game** and every **enemy aversion game** has a non-empty core.

5.2.6 The Stable Matching Problem

Stable Matching Problem

- Sets X (women) and Y (men) with $|X| = |Y| = n$.
- Each $x \in X$ has a strict preference order $\succ_x$ over Y; each $y \in Y$ has $\succ_y$ over X.
- Being unmatched is **least preferred** by every agent.
- A **matching** M is a set of pairs x–y where each person appears in at most one pair.
- A matching is **perfect** if $|M| = n$ (everyone is matched).

Blocking Pair

Given a perfect matching M, woman x and man y form a **blocking pair** if:

1. x prefers y to her current partner $M(x)$, **and**
2. y prefers x to his current partner $M(y)$.

Both can **jointly improve** by switching to each other.

Stable Matching

A **stable matching** is a perfect matching with **no blocking pairs**.

Running Example

Women	Men
Anna: Jacob $\succ$ Isaac $\succ$ Ken	Isaac: Anna $\succ$ Bella $\succ$ Clare
Bella: Isaac $\succ$ Jacob $\succ$ Ken	Jacob: Bella $\succ$ Anna $\succ$ Clare
Clare: Isaac $\succ$ Jacob $\succ$ Ken	Ken: Anna $\succ$ Bella $\succ$ Clare

$M_1 = \{A\text{–}K, B\text{–}J, C\text{–}I\}$: **Not stable**—Bella and Isaac form a blocking pair (Bella prefers Isaac over Jacob; Isaac prefers Bella over Clare).

$M_2 = \{A\text{–}I, B\text{–}J, C\text{–}K\}$: **Stable**—Isaac and Jacob have their top choices. Ken is matched to his least preferred, but no woman prefers Ken over her current partner.

Stable Matching as a Hedonic Game

An instance of stable matching is a hedonic game:

- Each man y prefers any pair $\{x, y\}$ (where x is a woman) to being alone.
- Preferences over pairs are determined by preferences over the opposite side.
- **Stable matching** = outcome in the **core** of this hedonic game.

5.2.7 The Stable Roommate Problem

Stable Roommate Problem

$2n$ people; each ranks all $2n - 1$ others. Assign roommate pairs with no blocking pairs.

Unlike the bipartite matching problem, stable outcomes **may not exist**.

Roommate Problem with No Stable Matching

Four agents with preferences:

- Adam: $B \succ C \succ D$
- Bob: $C \succ A \succ D$
- Chris: $A \succ B \succ D$
- Dan: $A \succ B \succ C$

- $\{A\text{–}D, B\text{–}C\}$: A–C is a blocking pair
- $\{A\text{–}C, B\text{–}D\}$: A–B is a blocking pair
- $\{A\text{–}B, C\text{–}D\}$: B–C is a blocking pair

No perfect matching is stable.

5.2.8 Gale-Shapley Deferred Acceptance Algorithm

DA Algorithm (Man-Proposing)

Each woman starts unmatched. The algorithm proceeds in rounds:

1. Each unmatched man proposes to the highest-ranked woman he has **not yet proposed to**.
2. Each woman who receives proposals **tentatively accepts** her most-preferred proposer (including her current tentative match) and **rejects** the rest.
3. Repeat until no new proposals are made.

Key invariant: Once a woman is matched, she never becomes unmatched—she only "trades up."

DA Terminates in $O(n^2)$ Rounds

Each man proposes to each woman at most once $\Rightarrow$ at most n^2 proposals total. Each round has at least one new proposal.

DA Produces a Perfect Matching

Proof: Suppose man y remains unmatched. Since $|X| = |Y|$, some woman x is also unmatched. But y must have proposed to all women—in particular to x. Once x received a proposal she becomes matched and never becomes unmatched. Contradiction. □

DA Outputs a Stable Matching

Proof: Let M be the output. Suppose x–y is not in M. Two cases:

- **Case 1:** y never proposed to $x \Rightarrow y$ is matched to someone he **prefers** to x.
- **Case 2:** y proposed to x and was rejected $\Rightarrow x$ is matched to someone she **prefers** to y.

In either case, x–y is **not** a blocking pair. □

Exam Tip: The DA "Master Theorem"

The DA algorithm terminates after $O(n^2)$ iterations and finds a stable matching for **any** problem instance. This was proved by Gale and Shapley (1962); Shapley and Roth received the 2012 Nobel Prize in Economics for this work.

5.2.9 Optimality and Pessimality

Valid Partner

Woman x is a **valid partner** of man y if there exists a stable matching M with x–$y \in M$.

Man-Optimal / Women-Pessimal Matching

- In a **man-optimal** matching, each man receives his **best valid partner**.
- In a **women-pessimal** matching, each woman receives her **worst valid partner**.

DA is Man-Optimal

The man-proposing DA outputs the **man-optimal** matching.

Proof (by contradiction):

1. Suppose some man is rejected by a valid partner during DA.
2. Let t be the **first** time this happens; let I be the man rejected by woman A in favour of man J.
3. Let M be a stable matching with A–$I \in M$, and let B be J's partner in M.
4. Since t is the first rejection by a valid partner, J has not been rejected by B yet $\Rightarrow$ J has not proposed to $B \Rightarrow J$ prefers A to B.
5. Also, A prefers J to I (she chose J).
6. $\Rightarrow$ A–J is a blocking pair in M—contradicting stability! □

DA is Women-Pessimal

The man-proposing DA outputs the **women-pessimal** matching.

Proof (by contradiction):

1. Suppose under DA, woman A is matched to man J, but J is **not** her worst valid partner.
2. Let I be her worst valid partner, so A prefers J to I.
3. Let M be a stable matching with A–$I \in M$, and B be J's partner in M.
4. Since DA is man-optimal, J prefers A to B.
5. $\Rightarrow$ A–J is a blocking pair in M—contradiction! □

Exam Tip: Symmetry of DA

Running DA with **women proposing** produces the **women-optimal, men-pessimal** stable matching. The two extremes may or may not coincide (they coincide iff the stable matching is unique).

5.2.10 Strategyproofness

Strategyproofness

An algorithm A is **strategyproof** if on every instance, no agent can benefit from misreporting their preferences. Formally, man y can **benefit from misreporting** if:

- Under truthful reports, A matches y to x.
- There exists a false ranking $\succ'$ such that reporting $\succ'$ (while others report truthfully) gives y a partner x' with $x' \succ_y x$.

Strategyproofness of DA

- The man-proposing DA is **strategyproof for men** (no man can benefit from lying).
- It is **not strategyproof for women**—women may benefit from misreporting.

Implication: No stable matching algorithm is strategyproof for **both** sides simultaneously.

5.2.11 Applications and Extensions

Practical Applications of DA

Domain	Application
University admissions	UCAS (UK), UAC (Australia)—>600k students annually
School choice	NYC and Boston public school placement
Medical residencies	NRMP (US)—∼30k graduates/year
Organ allocation	UK kidney sharing scheme; kidney exchange programs
Content delivery	CDNs matching users to edge servers by latency/bandwidth

Extension: Many-to-One Matching

Hospitals/schools have **quotas** (multiple positions). Stability: no pair x–y (school–student) such that y is not admitted, x prefers y to one current student, and y prefers x to their current school.

DA **extends directly** to this setting and produces a stable matching.

Extension: Unacceptable Pairs

Some agents may find certain matches unacceptable; $|X|$ and $|Y|$ need not be equal. Stability: no pair x–y where both find each other acceptable, x has a free slot or prefers y to a current student, and y is unmatched or prefers x to their current school.

DA **still works**.

Extension: Indifferences (Ties)

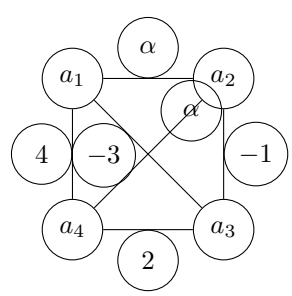
Preferences may have ties (e.g. schools are indifferent among students within the same priority group). **Weak stability:** no pair where both **strictly** prefer each other to current partners.

DA **still works** for this setting.

5.2.12 Worked Examples from Exercises

Worked Example 1: Stability Analysis of a Symmetric ASHG

Problem: Consider the symmetric ASHG (N, u) with $N = \{a_1, a_2, a_3, a_4\}$ and utilities given by the graph below, where $\alpha \in \mathbb{R}$:



Consider CS $= \{\{a_1, a_2, a_3\}, \{a_4\}\}$.

Step 1: Compute utilities in CS.

- $u_1(\{a_1, a_2, a_3\}) = v_1(a_2) + v_1(a_3) = \alpha + (-3) = \alpha - 3$

- $u_2(\{a_1, a_2, a_3\}) = v_2(a_1) + v_2(a_3) = \alpha + (-1) = \alpha - 1$
- $u_3(\{a_1, a_2, a_3\}) = v_3(a_1) + v_3(a_2) = -3 + (-1) = -4$
- $u_4(\{a_4\}) = 0$

(a) Individually rational if every player prefers their coalition to being alone ($u_i(S_i) \geq 0$):

- $u_1 = \alpha - 3 \geq 0 \Rightarrow \alpha \geq 3$
- $u_2 = \alpha - 1 \geq 0 \Rightarrow \alpha \geq 1$
- $u_3 = -4 < 0$ always!
- $u_4 = 0 \geq 0$ always.

$\Rightarrow$ CS is **never individually rational** (agent a_3 always prefers being alone).

(b) Nash stable if no player can benefit by moving to another coalition or forming a singleton. Since CS is never IR, it is **never Nash stable** (Nash stability implies IR).

(c) In the core if no group of players blocks. Since even singleton $\{a_3\}$ blocks (as a_3 prefers being alone), CS is **never in the core**.

Worked Example 2: DA Worst-Case Proposals

Problem: Construct an instance with n men and n women where each man makes at least $n - 1$ proposals under DA.

Construction: Let all men rank women identically: $w_1 \succ w_2 \succ \cdots \succ w_n$. Let woman w_i rank man m_i first (and the rest in any order).

Execution:

- **Round 1:** All n men propose to w_1. She accepts m_1 (her favourite), rejects $n - 1$ men.
- **Round 2:** Rejected men propose to w_2. She accepts m_2, rejects the rest.
- **Round k:** The $n - k + 1$ remaining men propose to w_k. She accepts m_k.
- Man m_n proposes to $w_1, w_2, \ldots, w_n$ (all n proposals). Man m_1 proposes once.

Man m_k makes k proposals. Total proposals: $1 + 2 + \cdots + n = \frac{n(n+1)}{2} = \Theta(n^2)$.

Each man m_k (for $k \geq 2$) makes at least $k \geq 2$ proposals; man m_n makes n proposals.

To ensure **each** man makes $\geq n - 1$ proposals, modify so that woman w_i ranks man m_{n-i+1} first (reverse order). Then every man proposes to all but one woman before being accepted.

Worked Example 3: Uniform Men's Preferences

Problem: All men rank women in the same order: $w_1 \succ w_2 \succ \cdots \succ w_n$. The i-th woman ranks m_i first. Describe the DA outcome.

Outcome: $M = \{w_i\text{–}m_i : 1 \leq i \leq n\}$ (each woman matched to her favourite man).

Why: By the analysis above, the algorithm proceeds round by round. In round k, all unmatched men propose to w_k. Woman w_k accepts m_k (her top choice) and rejects the rest. After n rounds, m_i is matched to w_i.

This is both the man-optimal and women-optimal stable matching (it is the **unique** stable matching).

5.2.13 Summary: Key Concepts

Concept	Key Fact
NTU game	Payoffs to individuals, not transferable; must track payoff vectors
Hedonic game	Player's utility depends only on coalition membership; outcome is a partition
ASHG	$v_i(S) = \sum_{j \in S \setminus \{i\}} v_i(j)$; directed graph representation
Nash stability	No unilateral move to another coalition; exists for symmetric ASHGs
CIS	Deviation needs approval from both joining and leaving coalitions; always exists
Core stability	No group can jointly deviate; may be empty even in symmetric ASHGs
Stable matching	Perfect matching with no blocking pairs
DA algorithm	$O(n^2)$, always finds stable matching; man-optimal, women-pessimal
Strategyproof	DA is strategyproof for proposers only
Stable roommates	One-sided variant; stable matching may not exist

6 Week 7: Fair Division (Part I)

6.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Explain foundational concepts of fair division: **proportionality**, **envy-freeness**, and **efficiency**.
2. Model resource allocation problems as fair division scenarios using mathematical and algorithmic frameworks.
3. Apply classic algorithms (**Cut-and-Choose**, **Dubins–Spanier**, **Even–Paz**, **Selfridge–Conway**) to achieve fair outcomes for **divisible goods**.
4. Analyse the strengths and limitations of different fair division procedures (guarantees, complexity, strategic aspects).
5. Assess allocation solutions for fairness and optimality using formal properties and counterexamples.
6. Solve practical and theoretical problems involving cake-cutting, rent division, and item allocation.

The Big Picture

The first half of the module covered strategic interaction (non-cooperative and cooperative games). The second half turns to **collective decision-making**: fair division (Weeks 7–8), mechanism design, and voting. Fair division asks: how should one or more goods be divided among agents so that every agent considers the outcome *fair*? This week focuses on **divisible goods** (cake-cutting); Week 8 treats **indivisible goods**.

What Examiners Like to Ask

- Define proportionality and envy-freeness; prove one implies the other for $n = 2$.
- Trace the **Cut-and-Choose**, **Dubins–Spanier**, or **Even–Paz** algorithm on a concrete instance.
- Show that a given algorithm is (or is not) proportional / envy-free / equitable.
- Analyse variants of moving-knife procedures (e.g. "1/3-envy-free" bounds).
- Compute the **price of proportionality** (welfare loss from requiring proportionality).
- Determine whether a connected allocation with given value guarantees exists.
- State complexity results under the **Robertson–Webb model**.

6.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Setting: agents, goods, divisible vs. indivisible
2. Fairness criteria: proportionality, envy-freeness, equitability, Pareto-optimality
3. Cake-cutting model and the Robertson–Webb computational model
4. Algorithm: Cut-and-Choose (2 agents, EF)
5. Algorithm: Dubins–Spanier moving knife (n agents, proportional)
6. Algorithm: Even–Paz (n agents, proportional, optimal complexity)

7. Algorithm: Selfridge–Conway (3 agents, EF)
8. General existence and complexity of envy-free algorithms
9. Worked examples and exam-style problems

6.2.1 The Fair Division Setting

Fair Division Problem

A **fair division problem** consists of:

- A finite set of **agents** $N = \{1, \ldots, n\}$.
- A set of **goods** (resources/items) to be divided.
- Each agent i has a **valuation function** v_i over (portions of) the goods.

An **allocation** assigns (parts of) the goods to agents.

Types of goods:

- **Divisible** (e.g. cake): any fraction can be allocated. This week's focus.
- **Indivisible** (e.g. a book): must be allocated whole. Week 8's focus.
- **Mixed**: indivisible goods + money (a divisible resource).

We assume goods are **static** (properties don't change), available in **single units**, and **cannot be shared** between agents.

6.2.2 Fairness Criteria

Let $(A_1, \ldots, A_n)$ be an allocation to n agents.

Proportionality

An allocation is **proportional** if every agent receives at least their "fair share":

$$v_i(A_i) \;\geq\; \frac{1}{n} \quad \text{for all } i \in N.$$

(Valuations are normalised so $v_i(\text{whole cake}) = 1$.)

Envy-Freeness (EF)

An allocation is **envy-free** if no agent prefers another agent's bundle:

$$v_i(A_i) \;\geq\; v_i(A_j) \quad \text{for all } i, j \in N.$$

EF $\Rightarrow$ Proportionality

For n agents, envy-freeness implies proportionality.

Proof. If the allocation is EF, then $v_i(A_i) \geq v_i(A_j)$ for all j. Summing over all j: $n \cdot v_i(A_i) \geq \sum_{j=1}^{n} v_i(A_j) = v_i(\text{whole cake}) = 1$, so $v_i(A_i) \geq 1/n$. □

Equivalence for $n = 2$

For two agents, an allocation is envy-free **if and only if** it is proportional.

Proof. EF $\Rightarrow$ proportional: shown above. Proportional $\Rightarrow$ EF: if $v_i(A_i) \geq 1/2$, then $v_i(A_j) = 1 - v_i(A_i) \leq 1/2 \leq v_i(A_i)$. $\square$

Exam Tip

For $n \geq 3$, proportionality does **not** imply envy-freeness. Be ready to give a counterexample (e.g. Dubins–Spanier chocolate/sponge example below).

Equitability

An allocation is **equitable** if all agents derive the same value: $v_i(A_i) = v_j(A_j)$ for all i, j.

Pareto Optimality

An allocation is **Pareto optimal** if there is no other allocation that makes some agent strictly better off without making any agent worse off.

6.2.3 Cake-Cutting Model

The "cake" is the interval $[0, 1]$. Each agent i has a valuation v_i that is:

- **Non-negative**: $v_i(X) \geq 0$ for all $X \subseteq [0, 1]$.
- **Additive**: $v_i(X \cup Y) = v_i(X) + v_i(Y)$ for disjoint X, Y.
- **Divisible**: for any X with $v_i(X) > 0$ and $0 < \lambda < 1$, there exists $Y \subseteq X$ with $v_i(Y) = \lambda \cdot v_i(X)$.
- **Normalised**: $v_i([0, 1]) = 1$.

Robertson–Webb Model

The complexity of cake-cutting algorithms is measured by the number of the following two queries:

1. **Evaluation query** $\mathrm{Eval}_i(x, y)$: returns $v_i([x, y])$ — "How much does agent i value piece $[x, y]$?"
2. **Cut query** $\mathrm{Cut}_i(x, \alpha)$: returns y such that $v_i([x, y]) = \alpha$ — "Starting at x, where must we cut so agent i values the piece at exactly α?"

6.2.4 Cut-and-Choose (2 Agents)

Cut-and-Choose Algorithm

1. Player 1 cuts the cake into two pieces X_1, X_2 with $v_1(X_1) = v_1(X_2) = 1/2$.
2. Player 2 chooses the piece they prefer; Player 1 gets the other.

Properties:

- **Envy-free** (and hence proportional for $n = 2$): Player 1 values both pieces equally, so never envies. Player 2 picks their favourite, so never envies.
- **Not necessarily equitable.**
- **Complexity**: 2 operations (1 cut + 1 eval) under Robertson–Webb.

Inequity in Cut-and-Choose

Cake: left half chocolate, right half sponge. Agent 1 values only chocolate ($v_1 = 1$ for chocolate half). Agent 2 values everything uniformly.

Agent 1 cuts first: Cuts chocolate half in two. Agent 2 picks one quarter (value 1/4 to them) + the sponge side is not offered this way. Actually: Agent 1 must cut so *they* value both halves equally. Since Agent 1 values only chocolate, they cut at the midpoint of the chocolate region, i.e. at $x = 1/4$. Pieces: $[0, 1/4]$ and $[1/4, 1]$.

- Agent 2 picks $[1/4, 1]$ (value 3/4). Agent 1 gets $[0, 1/4]$ (value 1/2).

Agent 2 cuts first: Cuts at $x = 1/2$ (values both halves equally). Agent 1 picks $[0, 1/2]$ (value 1 — all the chocolate!). Agent 2 gets $[1/2, 1]$ (value 1/2).

Both are envy-free, but the values differ: the cutter advantage varies by who cuts.

6.2.5 Dubins–Spanier Moving Knife (n Agents, Proportional)

Dubins–Spanier Algorithm

A referee moves a knife continuously from left to right across the cake.

1. When any player finds the piece to the left of the knife worth $\geq 1/n$ to them, they shout "Stop!"
2. That player receives the piece to the left, leaves the game. (Ties broken arbitrarily.)
3. Repeat with remaining $n - 1$ players on the remaining cake.
4. The last player receives the remainder.

Proportionality proof:

- Any player who shouts "Stop!" receives a piece worth $\geq 1/n$ to them.
- The last player never shouted (or tied): they valued each of the $n - 1$ removed pieces at $\leq 1/n$, so the remainder is worth $\geq 1 - (n - 1)/n = 1/n$.

Not envy-free for $n \geq 3$:

Dubins–Spanier Fails Envy-Freeness

Cake: $[0, 1/2]$ chocolate, $[1/2, 1]$ sponge. Three agents:

- Agent 1: values everything uniformly.
- Agents 2, 3: value only sponge.

Agent 1 shouts at 1/3, gets $[0, 1/3]$. Agents 2 and 3 both shout at 2/3 (the point where sponge value reaches 1/3). Say Agent 2 wins the tie, gets $[1/3, 2/3]$. Agent 3 gets $[2/3, 1]$.

Agent 2 **envies** Agent 3: Agent 3 has more sponge ($[2/3, 1]$ is all sponge, while $[1/3, 2/3]$ is half chocolate, half sponge).

Complexity: $\Theta(n^2)$ under Robertson–Webb (at each round, $O(n)$ queries for n rounds total: $n + (n - 1) + \cdots + 2 = \Theta(n^2)$).

Also not necessarily equitable or Pareto-optimal.

6.2.6 Even–Paz Algorithm (n Agents, Proportional, Optimal)

Even–Paz Algorithm

Divide-and-conquer approach (described for $n = 2^k$; extends to arbitrary n):

1. Let $[s, t]$ be the current piece. If $n = 1$: give it to the remaining agent.
2. Each agent i marks a point z_i on $[s, t]$ such that $v_i([s, z_i]) = \frac{1}{2}v_i([s, t])$.
3. Find the median mark z^* that splits the agents into two halves.
4. Recursively allocate $[s, z^*]$ to the left half of agents and $[z^*, t]$ to the right half.

Proportionality proof (by induction):

- **Base case** ($k = 0$): one agent gets the whole cake, valued at $1 \geq 1/n$.
- **Inductive step**: at step k, agents sharing a piece value it at $\geq 1/2^k$. At step $k + 1$, each agent is assigned to the half they value at $\geq \frac{1}{2} \cdot \frac{1}{2^k} = \frac{1}{2^{k+1}}$.
- After $\log n$ steps: each agent's piece is valued at $\geq 1/2^{\log n} = 1/n$.

Complexity: $O(n \log n)$ under Robertson–Webb ($\log n$ levels, $O(n)$ queries per level).

Optimality of Even–Paz

Any proportional cake-cutting algorithm requires $\Omega(n \log n)$ operations under the Robertson–Webb model. Hence Even–Paz is **optimal**.

Exam Tip: Comparing Proportional Algorithms

Algorithm	Agents	Guarantee	Complexity
Cut-and-Choose	2	EF (+ prop.)	$O(1)$
Dubins–Spanier	n	Proportional	$\Theta(n^2)$
Even–Paz	n	Proportional	$\Theta(n \log n)$

Dubins–Spanier is simpler but suboptimal in query complexity.

6.2.7 Selfridge–Conway Algorithm (3 Agents, Envy-Free)

Selfridge–Conway Algorithm

Achieves **envy-free allocation for 3 agents** in a bounded number of steps.

Stage 0 — Initial cut and trim:

1. Player 1 cuts the cake into 3 pieces they value equally (each worth 1/3).
2. Player 2 examines the pieces. If the two largest (in Player 2's view) are not equal, Player 2 **trims** the largest so the two largest are tied.
3. The trimmings become **Cake 2**; the three pieces form **Cake 1**.

Stage 1 — Allocate Cake 1:

1. Player 3 picks their favourite piece from Cake 1.
2. If Player 3 did *not* take the trimmed piece ⇒ Player 2 must take the trimmed piece.
   If Player 3 *did* take the trimmed piece ⇒ Player 2 picks their favourite from the remaining two.
3. Player 1 gets the remaining piece.

Let T = the player (2 or 3) who got the trimmed piece; T' = the other.

Stage 2 — Allocate Cake 2 (the trimmings):

1. Player T' cuts Cake 2 into 3 equal pieces (in their view).
2. Players choose in order: T, then Player 1, then T'.

Envy-freeness proof (per player, using additivity to argue on Cake 1 and Cake 2 separately):

- **Player** T': On Cake 2, divided it equally $\Rightarrow$ EF. On Cake 1: if $T' = 3$, they chose first among all three pieces $\Rightarrow$ EF. If $T' = 2$, they trimmed so the two largest pieces are equal, and they got one of those $\Rightarrow$ EF.
- **Player** T: On Cake 2, they choose first $\Rightarrow$ EF. On Cake 1, same argument as above.
- **Player 1**: On Cake 1, they cut equally so every piece is worth $1/3 \Rightarrow$ EF (they didn't get the trimmed piece). On Cake 2: Player 1 doesn't envy T' (Player 1 chose before T'). Player 1 doesn't envy T because T got the trimmed piece on Cake 1, which is worth $\leq 1/3$ to Player 1; even if T gets *all* of Cake 2, T's total $\leq 1/3$, so Player 1's "irrevocable advantage" ensures no envy.

6.2.8 General Envy-Free Cake Cutting

Existence (Brams–Taylor, 1995)

An envy-free cake-cutting algorithm exists for **any number of players** n under the Robertson–Webb model. However, its complexity cannot be bounded as a function of n alone (it depends on the valuation functions).

Bounded Protocol (Aziz–Mackenzie, 2016)

A **bounded** envy-free algorithm exists for n agents, but with extremely high complexity:

$$O(\underbrace{n^{n^{n^{\cdot^{\cdot^{\cdot^{n}}}}}}}_{5 \text{ levels}}).$$

Lower Bound (Procaccia, 2009)

Any envy-free cake-cutting algorithm requires $\Omega(n^2)$ queries.

Exam Tip: EF Complexity Summary

Agents	Algorithm	Complexity
$n = 2$	Cut-and-Choose	$O(1)$
$n = 3$	Selfridge–Conway	$O(1)$ (bounded)
$n \geq 4$	Aziz–Mackenzie	$O(n^{n^{n^{n^{n}}}})$
Lower bound (any n)		$\Omega(n^2)$

There is a large gap between the upper and lower bounds for $n \geq 4$.

6.2.9 Worked Examples

Example 1: Moving Knife Variant — 1/3-Envy-Freeness

Problem (Exercise 1). Consider a variant of the moving knife where agents shout "Stop!" when the left piece is worth exactly 1/3. The piece goes to the shouting agent, and the process continues. Prove the resulting allocation is 1/3-envy-free, i.e. $v_i(A_i) \geq v_i(A_j) - 1/3$ for all i, j.

Solution. An agent who shouts "Stop!" receives a piece worth exactly 1/3 to them. Any piece to the *left* of theirs was cut when some other agent valued it at 1/3; since the shouter waited, they value that earlier piece at $\leq 1/3$. So no envy towards earlier agents.

For pieces to the *right*: an agent might value a later piece at up to 2/3 (the entire remaining cake was worth 2/3 when they shouted). So the maximum envy is $v_i(A_j) - v_i(A_i) \leq 2/3 - 1/3 = 1/3$. Hence $v_i(A_i) \geq v_i(A_j) - 1/3$.

For the last agent(s) who didn't shout: they valued each removed piece at $\leq 1/3$, so the remainder is $\geq 1/3$, and the same bound applies.

Is 1/3 tight? Yes. Consider 2 agents, cake uniform. Agent 1 shouts at 1/3, gets $[0, 1/3]$ (value 1/3). Agent 2 gets $[1/3, 1]$ (value 2/3). Agent 1 envies Agent 2 by exactly $2/3 - 1/3 = 1/3$.

Example 2: Connected Allocations

Problem (Exercise 2). Two agents, Alice and Bob.

(a) Is there always a connected allocation with $v_A(A_A) \geq 0.6$ and $v_B(A_B) \geq 0.4$?
(b) What if we drop connectivity?
(c) Is there always a connected allocation where *one* agent gets ≥ 0.6 and the other ≥ 0.4?

Solution.

(a) No. Counterexample: both agents value the cake uniformly. Any connected allocation is a split at some point x: one gets $[0, x]$, the other $[x, 1]$. For Alice to get ≥ 0.6, she needs a piece of length ≥ 0.6, leaving Bob ≤ 0.4. But requiring $v_B \geq 0.4$ means Bob gets exactly 0.4 only if $x = 0.6$ or $x = 0.4$. This works for uniform valuations. However, consider: Alice values $[0, 0.5]$ at 1 and $[0.5, 1]$ at 0; Bob values $[0, 0.5]$ at 1 and $[0.5, 1]$ at 0. A connected split at x: Alice gets $[0, x]$, Bob gets $[x, 1]$. For Alice ≥ 0.6: need $x \geq 0.3$ (since $v_A([0, x]) = \min(2x, 1)$ for $x \leq 0.5$; so $2x \geq 0.6 \Rightarrow x \geq 0.3$). For Bob ≥ 0.4: need $v_B([x, 1]) \geq 0.4$, i.e. $v_B([x, 0.5]) = 2(0.5 - x) \geq 0.4 \Rightarrow x \leq 0.3$. So $x = 0.3$ works: Alice gets 0.6, Bob gets 0.4. This particular example works, but we can also swap who gets which side.

Actually, for the specific question: with *arbitrary* valuations and a *fixed assignment* (Alice gets left, Bob gets right), it is **not always** possible. Consider Alice values $[0.5, 1]$ at 1 and Bob values $[0, 0.5]$ at 1. Any connected split $[0, x]$ to Alice, $[x, 1]$ to Bob: Alice needs $v_A([0, x]) \geq 0.6 \Rightarrow x > 0.5$, so Bob gets $[x, 1]$ with $x > 0.5$, but $v_B([x, 1]) = 0 < 0.4$.

(b) Yes (without connectivity). By a probabilistic / measure-theoretic argument, for any two additive valuations summing to 1, we can find a (possibly disconnected) allocation giving each agent $\geq 1/2$, which exceeds both 0.6 and 0.4... Actually, we need Alice ≥ 0.6 *and* Bob ≥ 0.4. Since EF gives each $\geq 1/2 > 0.4$, but Alice needs 0.6. Without connectivity, we can partition $[0, 1]$ into intervals and assign optimally. The answer is **yes**: we can always achieve $v_A \geq 0.6$ and $v_B \geq 0.4$ without connectivity (this follows from Lyapunov's theorem on the range of vector measures).

(c) Yes. We can always find a connected allocation where one agent gets ≥ 0.6 and the other gets ≥ 0.4. Consider the function $f(x) = v_A([0, x])$: it is continuous, $f(0) = 0$, $f(1) = 1$. By IVT, there exists x^* with $f(x^*) = 0.6$. Then $v_A([0, x^*]) = 0.6$ and $v_B([x^*, 1]) = 1 - v_B([0, x^*])$. If $v_B([x^*, 1]) \geq 0.4$, we are done. Otherwise, $v_B([0, x^*]) > 0.6$, so assign Bob $[0, x^*]$ (value $> 0.6 \geq 0.4$). By IVT, find y^* with $v_B([0, y^*]) = 0.4$; then give Alice $[y^*, 1]$ with $v_A([y^*, 1]) = 1 - v_A([0, y^*])$. Since $y^* < x^*$ (Bob reaches 0.4 before Alice reaches 0.6), $v_A([y^*, 1]) > 0.4$. The flexibility of choosing *which* agent gets which side guarantees existence.

Example 3: Price of Proportionality

Problem (Exercise 3). $n = k^2$ agents. Agents $1, \ldots, k$ have valuation uniform on $[(i-1)/k,\ i/k]$ and 0 elsewhere (agent i values subintervals of $[(i-1)/k, i/k]$ at k times their length). The remaining $k^2 - k$ agents value the entire cake uniformly.

(a) Maximum social welfare. Give each "specialist" agent i (for $i = 1, \ldots, k$) their favourite interval $[(i-1)/k, i/k]$. Each gets value 1. The remaining $k^2 - k$ uniform agents get nothing (value 0).

$$\text{Max SW} = k \cdot 1 + (k^2 - k) \cdot 0 = k.$$

(b) Maximum social welfare under proportionality. Each of the $n = k^2$ agents must get value $\geq 1/k^2$. The $k^2 - k$ uniform agents need total cake length $\geq (k^2 - k)/k^2 = 1 - 1/k$. This leaves $\leq 1/k$ of the cake for the k specialist agents combined. Since the specialists' favourite intervals are disjoint and each has length $1/k$, at most one specialist can get their full interval (impossible—we've only $1/k$ total). In fact, each specialist must get at least $1/k^2$ (proportional share) from their interval, contributing $k \cdot (1/k^2) = 1/k$ to specialist welfare at minimum.

The uniform agents' total welfare from their share: each gets $\geq 1/k^2$ cake length, so total uniform welfare $\geq (k^2 - k)/k^2$. The specialists' welfare: each specialist i gets some portion of $[(i-1)/k, i/k]$ with total specialist cake $\leq 1/k$, so total specialist welfare $\leq k \cdot 1 = k$ is not achievable.

Optimally: give each specialist exactly $1/k^2$ of their interval (satisfying proportionality, value $= k \cdot 1/k^2 = 1/k$), and distribute the rest $(1 - k \cdot 1/k^2 = 1 - 1/k)$ to uniform agents. Uniform agents sharing $1 - 1/k$ cake, each needing $\geq 1/k^2$, total uniform welfare $= 1 - 1/k$.

$$\text{Max proportional SW} = k \cdot \frac{1}{k} + (1 - \frac{1}{k}) = 1 + 1 - \frac{1}{k} = 2 - \frac{1}{k}.$$

Price of proportionality $= k/(2 - 1/k) = k^2/(2k - 1) \approx k/2$ for large k, i.e. $\approx \sqrt{n}/2$.

Example 4: Cut-and-Choose Trace

Problem. Cake: $[0, 1]$, left half strawberry, right half vanilla. Agent A values strawberry at 0.8, vanilla at 0.2. Agent B values both equally. Agent A cuts. Where do they cut, and what does each agent get?

Solution. Agent A wants two pieces of equal value (0.5 each). Strawberry density for A: $0.8/0.5 = 1.6$ per unit length on $[0, 0.5]$; vanilla density: $0.2/0.5 = 0.4$ on $[0.5, 1]$.

Cut at point x. Value of $[0, x]$ to A:

- If $x \leq 0.5$: $v_A = 1.6x$. Set $= 0.5$: $x = 5/16 = 0.3125$.

So A cuts at $x = 0.3125$. Piece 1: $[0, 0.3125]$, Piece 2: $[0.3125, 1]$.

Agent B (uniform): $v_B(\text{Piece 1}) = 0.3125$, $v_B(\text{Piece 2}) = 0.6875$. B picks Piece 2.

Result: A gets $[0, 0.3125]$ (value 0.5), B gets $[0.3125, 1]$ (value 0.6875). Envy-free. Not equitable.

Example 5: Even–Paz Trace

Problem. Four agents with the following valuations on $[0, 1]$: Agent 1 uniform, Agent 2 values $[0, 0.5]$ at 1, Agent 3 values $[0.5, 1]$ at 1, Agent 4 uniform. Run Even–Paz.

Solution. Each agent marks the midpoint of their value on $[0, 1]$:

- Agent 1: $z_1 = 0.5$ (uniform $\Rightarrow$ midpoint by length).

- Agent 2: $z_2 = 0.25$ ($v_2([0, 0.25]) = 0.5$).
- Agent 3: $z_3 = 0.75$ ($v_3([0.5, 0.75]) = 0.5$, but need $v_3([0, z_3]) = 0.5$; since $v_3([0, 0.5]) = 0$, need $v_3([0.5, z_3]) = 0.5 \Rightarrow z_3 = 0.75$).
- Agent 4: $z_4 = 0.5$.

Sorted marks: $0.25, 0.5, 0.5, 0.75$. Median split $z^* = 0.5$. Left group (marks ≤ 0.5): Agents $\{2, 1, 4\}$—but we need exactly 2 per side. Take the two smallest marks: Agents 2 and 1 go left on $[0, 0.5]$; Agents 4 and 3 go right on $[0.5, 1]$.

Recurse on each half with 2 agents each (essentially Cut-and-Choose). Each agent ends up with $\geq 1/4$ of total value.

Example 6: Selfridge–Conway Trace

Problem. Three agents. Agent 1 values the cake uniformly. Agent 2 values $[0, 0.5]$ at 0.8 and $[0.5, 1]$ at 0.2. Agent 3 values $[0, 0.5]$ at 0.3 and $[0.5, 1]$ at 0.7.

Stage 0: Agent 1 cuts into three equal pieces: $[0, 1/3], [1/3, 2/3], [2/3, 1]$, each worth $1/3$ to Agent 1.

Agent 2's valuations: $v_2([0, 1/3]) = 0.8 \cdot (2/3) = 0.533$, $v_2([1/3, 1/2]) = 0.8 \cdot (1/3) = 0.267$, so $v_2([1/3, 2/3]) = 0.267 + 0.2 \cdot (1/6) = 0.267 + 0.033 = 0.3$, $v_2([2/3, 1]) = 0.2 \cdot (1/3)/0.5 = 0.133$. Largest piece for Agent 2: $[0, 1/3]$ at 0.533, second largest: $[1/3, 2/3]$ at 0.3.

Agent 2 trims $[0, 1/3]$ to match 0.3. The trimming (Cake 2) has value $0.533 - 0.3 = 0.233$ to Agent 2.

Stage 1: Agent 3 picks their favourite from the three Cake 1 pieces. Then allocation proceeds per the algorithm. **Stage 2:** Cake 2 divided by T', chosen in order $T, 1, T'$. Result is envy-free.

7 Week 8: Fair Division (Part II)

7.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

1. Define the **indivisible goods allocation problem**: items, agents, valuation functions, bundles, complete vs. partial allocations.
2. Distinguish **additive** and **monotone** valuations and explain why indivisibility makes perfect fairness unattainable.
3. State and apply relaxed fairness notions: **EF1** (envy-freeness up to one good) and **PROP1** (proportionality up to one good).
4. Execute the **round-robin algorithm** for additive valuations and prove it guarantees EF1.
5. Execute the **envy-cycle elimination (ECE) algorithm** for monotone valuations and prove it guarantees EF1.
6. Define **utilitarian**, **egalitarian**, and **Nash** welfare; compare their fairness and efficiency properties.
7. State that **maximum Nash welfare** $\Rightarrow$ EF1 + Pareto optimality.
8. Define **EFX** (envy-freeness up to any good) and discuss its existence as an open problem.

The Big Picture

Week 7 covered fair division of **divisible** resources (cake-cutting). This week moves to the harder setting where goods are **indivisible**—they cannot be split. Because perfect envy-freeness may be impossible (e.g. one item, two agents), we study **relaxed fairness notions** (EF1, PROP1, EFX) and algorithms that guarantee them. We also explore the tension between **fairness** and **efficiency**, culminating in the result that maximising **Nash welfare** achieves both EF1 and Pareto optimality.

What Examiners Like to Ask

- Trace the **round-robin algorithm** on a given instance and verify EF1.
- Construct an example where round-robin is **not Pareto efficient** or **not strategyproof**.
- Trace the **envy-cycle elimination algorithm**: build the envy graph, identify cycles, perform reallocations.
- Given an allocation, check whether it is **EF1**, **EFX**, or **Pareto optimal**.
- Compare **utilitarian**, **egalitarian**, and **Nash welfare** on a given instance.
- Explain why **Nash welfare maximisation** implies EF1 + Pareto optimality.
- Prove that round-robin guarantees each agent $\geq \frac{2}{3}$ of their MMS under certain conditions.
- Argue existence of **MMS allocations** when $|G| \leq |N| + 1$.

7.2 Overview + Core Concepts + Extended Theory

Roadmap

1. Indivisible goods: model, valuations, allocations
2. Fairness relaxations: EF1 and PROP1

3. Round-robin algorithm (additive valuations)
4. Envy graph and envy-cycle elimination (monotone valuations)
5. Welfare concepts: utilitarian, egalitarian, Nash
6. Fairness vs. efficiency trade-offs
7. Envy-freeness up to any good (EFX)

7.2.1 Indivisible Goods: Model and Applications

Indivisible Goods Allocation Problem

An instance consists of:

- A set $G = \{g_1, \ldots, g_m\}$ of m **indivisible goods**.
- A set $N = \{1, \ldots, n\}$ of n **agents**.
- Each agent i has a **valuation function** $v_i : 2^G \to \mathbb{R}$ (or $\mathbb{Z}$) with $v_i(\emptyset) = 0$.

An **allocation** $A = (A_1, \ldots, A_n)$ is a partition of a subset of G into n bundles with $A_i \cap A_j = \emptyset$ for $i \neq j$.

- **Complete**: $\bigcup_{i \in N} A_i = G$ (all goods allocated).
- **Partial**: $\bigcup_{i \in N} A_i \subseteq G$ (some goods unallocated).

Additive Valuations

Valuations are **additive** if for every agent i and every $S \subseteq G$:

$$v_i(S) = \sum_{g \in S} v_i(\{g\}).$$

Monotone Valuations

Valuations are **monotone** if for every agent i and every $S \subseteq T \subseteq G$:

$$v_i(S) \leq v_i(T).$$

Adding items never decreases value. Note: additive valuations with non-negative values are always monotone.

Applications: dividing inheritance among heirs, allocating tasks to team members, assigning dormitory rooms to students, distributing computing resources in data centres.

7.2.2 Fairness Relaxations: EF1 and PROP1

Perfect envy-freeness (EF) may not exist for indivisible goods (e.g. 1 item, 2 agents). We relax:

Envy-Freeness up to One Good (EF1)

An allocation A is **EF1** if for every pair of agents $i, j \in N$ with $A_j \neq \emptyset$, there exists some item $g \in A_j$ such that:

$$v_i(A_i) \geq v_i(A_j \setminus \{g\}).$$

Intuition: agent i may envy j, but the envy disappears after removing **some single item** from j's bundle.

Proportionality up to One Good (PROP1)

An allocation A is **PROP1** if for every agent i there exists some $g \in G \setminus A_i$ such that:

$$v_i(A_i \cup \{g\}) \geq \frac{v_i(G)}{n}.$$

EF1 vs EF vs EFX

The strength hierarchy is: EF $\implies$ EFX $\implies$ EF1. For indivisible goods, EF may not exist, so EF1 is the standard achievable guarantee.

7.2.3 Round-Robin Algorithm (Additive Valuations)

Round-Robin Procedure

Fix an ordering of agents $1, 2, \ldots, n$. Agents take turns picking their most preferred remaining item:

- Agent i picks in rounds $i,\ n+i,\ 2n+i,\ \ldots$
- Each agent always picks the remaining item with highest value to them.

Claim: Round-robin always produces an **EF1 allocation** for additive valuations.

Proof sketch. Consider agents i and j:

- *Case 1: i picks before j.* In each round, i picks before j, so i's k-th item $\geq_i$ j's k-th item. By additivity, $v_i(A_i) \geq v_i(A_j)$, so i does not envy j at all.
- *Case 2: j picks before i.* j's first pick may beat i's, but from the second round onward, i picks before j. So i's first item $\geq_i$ j's second item, i's second item $\geq_i$ j's third item, etc. Removing j's **first item** eliminates envy. □

Round-Robin Trace

Three agents, five items. Valuations (additive):

	g_1	g_2	g_3	g_4	g_5
a_1	5	8	4	6	2
a_2	3	4	7	2	5
a_3	6	3	5	4	1

Order: a_1, a_2, a_3.

- Round 1: a_1 picks g_2 (val 8), a_2 picks g_3 (val 7), a_3 picks g_1 (val 6).
- Round 2: a_1 picks g_4 (val 6), a_2 picks g_5 (val 5).

Result: $A_1 = \{g_2, g_4\}$, $A_2 = \{g_3, g_5\}$, $A_3 = \{g_1\}$.

Check EF1: a_3 envies a_1 ($v_3(A_1) = 7 > 6$), but removing g_4: $v_3(\{g_2\}) = 3 \leq 6 = v_3(A_3)$. ✓

7.2.4 Limitations of Round-Robin

Not Pareto efficient. Consider two agents, four items:

	g_1	g_2	g_3	g_4
a_1	10	9	8	7
a_2	100	3	2	1

Round-robin (a_1 first): $A_1 = \{g_1, g_3\}$ (val 18), $A_2 = \{g_2, g_4\}$ (val 4). But swapping g_1 for $\{g_2, g_4\}$: a_1 gets $\{g_2, g_3, g_4\} \to$ val 24, a_2 gets $\{g_1\} \to$ val 100. Both strictly improve $\Rightarrow$ Pareto dominated.

Not strategyproof. Two agents, four items:

	g_1	g_2	g_3	g_4
a_1	9	8	2	1
a_2	1	9	2	3

Truthful (a_1 first): a_1 picks $g_1, g_3 \to$ val 11. But if a_1 *misreports* and picks g_2 first, then a_2 takes g_4, and a_1 takes g_1 in round 2: a_1 gets $\{g_1, g_2\} \to$ val $17 > 11$.

7.2.5 Envy-Cycle Elimination (Monotone Valuations)

Round-robin requires additive valuations (comparing individual items). For general **monotone** valuations, we use the **envy-cycle elimination** (ECE) algorithm.

Envy Graph

Given an allocation A, the **envy graph** $\mathcal{G}(A)$ has:

- Vertices: agents N.
- Directed edge $i \to j$ iff $v_i(A_j) > v_i(A_i)$ (agent i envies agent j).

Reallocation Along a Cycle (RAC)

If the envy graph contains a directed cycle $i_1 \to i_2 \to \cdots \to i_k \to i_1$, we **shift bundles along the cycle**: agent i_1 receives A_{i_2}, agent i_2 receives A_{i_3}, …, agent i_k receives A_{i_1}.

Each agent on the cycle gets a bundle they **strictly prefer**, so the number of edges in the envy graph strictly decreases.

ECE Algorithm

Starting from the empty allocation, repeat:

1. While the envy graph has a **cycle**: perform a RAC step (shift bundles along cycle).
2. Once the envy graph is **acyclic**: find an **unenvied agent** (a source in the DAG) and give them one unallocated item.

Terminates in polynomial time and produces an **EF1 allocation** for monotone valuations.

Why it works:

- An acyclic graph always has a source (unenvied agent). Giving an unallocated item to an unenvied agent preserves EF1 by monotonicity.
- RAC steps preserve EF1 (just permuting bundles; each cycle agent improves).

- **Potential function:** each agent scores their bundle by $n - \text{rank}(A_i)$. Total score $\in [0, n(n-1)]$, strictly increases with each RAC step $\Rightarrow$ at most $O(n^2)$ RAC steps between item allocations.
- Total: m item-allocation steps $\times O(n^2)$ RAC steps each $\Rightarrow$ polynomial time.

Envy-Cycle Elimination Trace

Three agents, allocation: $A_1 = \{g_1, g_5\}$, $A_2 = \{g_2, g_3\}$, $A_3 = \{g_4\}$.

Suppose the envy graph has cycle $a_1 \to a_2 \to a_3 \to a_1$. Perform RAC:

- a_1 gets $A_2 = \{g_2, g_3\}$, a_2 gets $A_3 = \{g_4\}$, a_3 gets $A_1 = \{g_1, g_5\}$.

Each cycle agent now has a bundle they preferred $\Rightarrow$ envy edges within the cycle decrease. Repeat until acyclic, then allocate the next unallocated item to an unenvied agent.

ECE vs Round-Robin

ECE is **more general** (works for monotone valuations, not just additive). Both guarantee EF1 but neither guarantees Pareto optimality.

7.2.6 Concepts of Welfare

Welfare Objectives

Given allocation $A = (A_1, \ldots, A_n)$:

Welfare	Formula	Intuition
Utilitarian	$\sum_{i \in N} v_i(A_i)$	Maximise total happiness
Egalitarian	$\min_{i \in N} v_i(A_i)$	Maximise the worst-off agent
Nash	$\prod_{i \in N} v_i(A_i)$	Balance total and worst-off

Welfare Comparison

Five items, three agents with uniform per-item values: $v_1(g) = 10$, $v_2(g) = 1$, $v_3(g) = 3$ for all g.

- **Utilitarian max:** give all 5 items to $a_1 \Rightarrow$ welfare $= 50$. Highly unfair.
- **Egalitarian max:** give 3 items to a_2, 1 each to $a_1, a_3 \Rightarrow$ min utility $= 3$. Not EF1 (a_1 envies a_2 even after removing any one item from a_2).
- **Nash max:** give 2 items to two agents, 1 to one $\Rightarrow$ product $= 120$ regardless of assignment. Achieves EF1.

7.2.7 Fairness vs. Efficiency Trade-Offs

Utilitarian Welfare $\Rightarrow$ Pareto Optimality

Any allocation maximising utilitarian welfare is Pareto optimal. *Proof:* a Pareto improvement would increase the sum, contradicting maximality.

However, utilitarian welfare can be **highly unfair** (may assign all items to one agent).

Egalitarian Welfare: Limitations

Maximising egalitarian welfare:

- Is **not necessarily EF1**.
- Is **not necessarily Pareto optimal**. Example: 2 agents, 3 items a, b, c. Agent 1: $v_1(a) = v_1(b) = x$, $v_1(c) = 0$. Agent 2: $v_2(a) = v_2(c) = x$, $v_2(b) = 0$. Egalitarian max gives $A_1 = \{a\}$, $A_2 = \{b, c\}$ (min $= x$), but giving c to agent 1 improves them without hurting agent 2.

Nash Welfare ⇒ EF1 + Pareto Optimality

For additive valuations, any allocation maximising the **Nash welfare** (product of utilities) is both:

- **Pareto optimal** (same argument as utilitarian: a Pareto improvement increases the product).
- **EF1** (proof is non-trivial; beyond the scope of this course).

This makes Nash welfare the "best of both worlds" objective.

Nash Welfare and Scaling

Nash welfare is **scale-invariant** per agent: scaling v_i by a constant $c > 0$ does not change the maximiser. This is because $\arg\max \prod c \cdot v_i(A_i) = \arg\max\, c \cdot \prod v_i(A_i) = \arg\max \prod v_i(A_i)$.

7.2.8 Envy-Freeness up to Any Good (EFX)

Envy-Freeness up to Any Good (EFX)

An allocation A is **EFX** if for every pair i, j with $A_j \neq \emptyset$ and for **every** item $g \in A_j$:

$$v_i(A_i) \geq v_i(A_j \setminus \{g\}).$$

Contrast with EF1: EF1 requires $\exists\, g$; EFX requires $\forall\, g$.

EFX: What Is Known

- **2 agents**, monotone valuations: EFX exists.
- **3 agents**, additive valuations: EFX exists.
- ≥ 4 **agents**, additive valuations: **open problem** — existence is unknown.
- **Identical additive valuations**: EFX exists and can be computed in polynomial time.

EFX for Identical Additive Valuations

When all agents share the same additive valuation v:

1. Sort items by decreasing value: $v(g_1) \geq v(g_2) \geq \cdots \geq v(g_m)$.
2. Assign each item in turn to the agent with the **smallest current bundle value**.

Proof of EFX: Suppose agent i envies agent j, and let g be the **last item** j received. At the moment j received g, agent i's bundle was worth $\geq j$'s bundle (since i was at least as satisfied). So $v(A_i) \geq v(A_j \setminus \{g\})$.

Since g was the *smallest-value* item in A_j (items assigned in decreasing order to the least-satisfied agent), removing *any* item from A_j removes at least as much value as removing g. Hence $v(A_i) \geq v(A_j \setminus \{g'\})$ for all $g' \in A_j$. □

7.2.9 Worked Examples from Exercises

Nash Welfare and Scaling (Exercise 1)

Problem: Let $(A_1, \ldots, A_n)$ maximise Nash welfare. Agent i's valuation is scaled: $v_i'(g) = 10 \cdot v_i(g)$, others unchanged. Show the same allocation maximises Nash welfare for the modified instance.

Solution: The Nash welfare of allocation A under the modified instance is:

$$\mathrm{NW}'(A) = v_i'(A_i) \cdot \prod_{j \neq i} v_j(A_j) = 10 \cdot v_i(A_i) \cdot \prod_{j \neq i} v_j(A_j) = 10 \cdot \mathrm{NW}(A).$$

Since every allocation's Nash welfare is multiplied by the **same constant** 10, the maximiser does not change. The allocation that maximised NW also maximises $\mathrm{NW}' = 10 \cdot \mathrm{NW}$.

MMS Allocation When $|G| \leq |N| + 1$ (Exercise 2)

Problem: Show that when $|G| \leq |N| + 1$ and valuations are additive, an MMS allocation always exists.

Solution: Recall that agent i's **maximin share** MMS_i is the maximum over all partitions $(B_1, \ldots, B_n)$ of G into n bundles of $\min_k v_i(B_k)$.

With $|G| \leq n + 1$ items and n agents, any partition into n bundles has at most one bundle with 2 items and the rest with ≤ 1 item each (by pigeonhole).

Consider the MMS-defining partition for agent i. The minimum bundle has value MMS_i, and it contains at most 1 item (at most one bundle gets 2 items, so the worst bundle gets ≤ 1).

Now allocate greedily: repeatedly find an unallocated item worth $\geq \mathrm{MMS}_i$ to some unassigned agent i and assign it. With so few items relative to agents, each agent can receive at least one item meeting their MMS threshold (the details follow from a careful pigeonhole argument on the partition structure).

Key insight: When items are scarce ($|G| \leq n + 1$), MMS partitions have mostly singletons, making it easy to satisfy each agent's guarantee.

Round-Robin and $\frac{2}{3}$-MMS Guarantee (Exercise 3)

Problem: Each agent has additive valuations with $v_i(G) = 1$ and $v_i(\{g\}) \leq \frac{1}{3n}$ for all g. Show round-robin guarantees each agent at least $\frac{2}{3} \cdot \mathrm{MMS}_i$.

Solution: Since $v_i(G) = 1$ and valuations are additive, by a pigeonhole/averaging argument, $\mathrm{MMS}_i \leq \frac{1}{n}$ (the best you can guarantee when dividing into n bundles is the average).

In round-robin with n agents, agent i picks in positions $i, n + i, 2n + i, \ldots$ In each of their turns, they pick the most valuable remaining item. Since each item is worth $\leq \frac{1}{3n}$, at most $\frac{1}{3n}$ of value is "lost" per pick by other agents in each round.

Agent i picks at least $\lfloor m/n \rfloor$ items. In each round, agent i picks an item at least as valuable as the average remaining item. Across all rounds, the cumulative value agent i collects is at least $\frac{1}{n} - \frac{(n-1)}{3n} \cdot$ (rounds) $\cdot$ (max item value), which can be bounded below by $\frac{2}{3n} \geq \frac{2}{3} \cdot \mathrm{MMS}_i$.

Key insight: The small item-value assumption ($\leq \frac{1}{3n}$) ensures no single item dominates, so round-robin distributes value almost evenly.

7.2.10 Summary: Algorithms and Guarantees

Algorithm/Objective	**Valuations**	**EF1**	**Pareto Opt.**	**Strategyproof**
Round-Robin	Additive	✓	×	×
Envy-Cycle Elim.	Monotone	✓	×	×
Max Utilitarian	Additive	×	✓	—
Max Egalitarian	Additive	×	×	—
Max Nash Welfare	Additive	✓	✓	—